

Sparsity-Aware Distributed Learning for Gaussian Processes With Linear Multiple Kernel

Richard Cornelius Suwandi^{ID}, *Graduate Student Member, IEEE*, Zhidi Lin^{ID}, *Graduate Student Member, IEEE*, Feng Yin^{ID}, *Senior Member, IEEE*, Zhiguo Wang^{ID}, *Member, IEEE*, and Sergios Theodoridis^{ID}, *Life Fellow, IEEE*

Abstract—Gaussian processes (GPs) stand as crucial tools in machine learning and signal processing, with their effectiveness hinging on kernel design and hyperparameter optimization. This article presents a novel GP linear multiple kernel (LMK) and a generic sparsity-aware distributed learning framework to optimize the hyperparameters. The newly proposed grid spectral mixture product (GSMP) kernel is tailored for multidimensional data, effectively reducing the number of hyperparameters while maintaining good approximation capability. We further demonstrate that the associated hyperparameter optimization of this kernel yields sparse solutions. To exploit the inherent sparsity of the solutions, we introduce the sparse linear multiple kernel learning (SLIM-KL) framework. The framework incorporates a quantized alternating direction method of multipliers (ADMMs) scheme for collaborative learning among multiple agents, where the local optimization problem is solved using a distributed successive convex approximation (DSCA) algorithm. SLIM-KL effectively manages large-scale hyperparameter optimization for the proposed kernel, simultaneously ensuring data privacy and minimizing communication costs. The theoretical analysis establishes convergence guarantees for the learning framework, while experiments on diverse datasets demonstrate the superior prediction performance and efficiency of our proposed methods.

Index Terms—Communication-efficient distributed learning, Gaussian process (GP), linear multiple kernel (LMK), multidimensional data, sparsity-aware learning.

NOMENCLATURE

n	Training size.
P	Input dimension of the data.
Q	Number of hyperparameters (grid points).
N	Number of local agents.
s	Number of computing units.

Received 17 January 2024; revised 2 June 2024, 21 August 2024, 21 October 2024, and 5 January 2025; accepted 14 January 2025. Date of publication 28 January 2025; date of current version 6 August 2025. This work was supported in part by NSFC under Grant 62271433 and in part by Shenzhen Science and Technology Program under Grant JCYJ20220530143806016 and RCJC20210609104448114. An earlier version of this paper was presented at the IEEE International Conference on Information Fusion (FUSION), 2022 [DOI: 10.23919/FUSION49751.2022.9841347]. (Richard Cornelius Suwandi and Zhidi Lin are co-first authors.) (Corresponding author: Feng Yin.)

Richard Cornelius Suwandi and Feng Yin are with the School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen 518172, China (e-mail: richardsuwandi@link.cuhk.edu.cn; yinfeng@cuhk.edu.cn).

Zhidi Lin was with the School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen 518172, China. He is now with the Department of Statistics and Data Science, National University of Singapore, Singapore 117546 (e-mail: zhidilin@nus.edu.sg).

Zhiguo Wang is with the Department of Mathematics, Sichuan University, Chengdu, Sichuan 610064, China (e-mail: wangzhiguo@scu.edu.cn).

Sergios Theodoridis is with the Department of Informatics and Telecommunications, National and Kapodistrian University of Athens, 115 28 Athens, Greece (e-mail: stheodor@di.uoa.gr).

Digital Object Identifier 10.1109/TNNLS.2025.3531784

θ	Global hyperparameters (weights of grid spectral mixture product (GSMP) kernel).
ξ_j	Local hyperparameters of the j th agent.
$\mathcal{D}_j$	Local dataset owned by the j th agent.
l	Objective function [negative log marginal likelihood of the Gaussian process (GP)].
$\tilde{l}$	Surrogate function for the objective.
λ_j	Dual variable in alternating direction method of multiplier (ADMM).
ρ_j	Penalty parameter in ADMM.
$\mathcal{Q}$	Stochastic quantization function.
Δ	Quantization resolution.

I. INTRODUCTION

RECENTLY, Bayesian methods have gained significant attention across a wide range of machine learning and signal processing applications. They are known for their ability to enhance model robustness by leveraging prior knowledge and effectively handling uncertainties associated with the estimates [2]. Among these methods, the GP has emerged as a particularly promising model that offers reliable estimates accompanied with uncertainty quantifications [3], [4], [5]. The distinctive strength of GP lies in its explicit modeling of the underlying function using a GP prior, which empowers GP to capture intricate patterns in the data, even with limited observations [6], [7]. Unlike conventional machine learning models that rely on point estimates, GP embraces a probabilistic framework that yields a posterior distribution over the function given the observed data [8], [9].

The representation power of a GP model heavily relies on the selection of an appropriate kernel function. Several traditional kernel functions have been used in GP models, including the squared-exponential (SE) kernel, Ornstein–Uhlenbeck kernel, rational quadratic kernel, and periodic kernel, among others [3]. These kernel functions can even be combined, for example, into a linear multiple kernel (LMK) in the form of a linearly weighted sum to enhance the overall modeling capacity [10]. However, the selection of these preliminary kernel functions often relies heavily on subjective expert knowledge, making it impractical for complex applications [11].

To address the challenge of manual kernel design, Lázaro-Gredilla et al. [12] introduced the sparse spectrum GP (SSGP) that employs a sparse set of spectral points to approximate the spectral density of a stationary kernel function. However, optimizing these spectral points can lead to overfitting, especially as their number increases, resulting in poor generalization on unseen data [13]. Wilson and Adams [14] proposed the

spectral mixture (SM) kernel to discover underlying data patterns automatically via formulating the spectral density of kernel as a Gaussian mixture. However, the original SM kernel presents difficulties in tuning its hyperparameters due to the nonconvex nature of the optimization problem. This increases the risk of getting stuck at a suboptimal solution, especially when dealing with a large number of hyperparameters. The grid SM (GSM) kernel was proposed to address this drawback in the SM kernel [15]. Specifically, the GSM kernel focuses on optimizing the weights of each kernel component while fixing other hyperparameters, resulting in a more favorable optimization structure. Consequently, the GSM kernel acts as an LMK, allowing its subkernels to exhibit beneficial low-rank properties under reasonable conditions [15]. By identifying a sparse subset of the most relevant frequency components in the input data, the GSM kernel enhances the model interpretability and facilitates a clearer understanding of the underlying data patterns.

While employing GSM kernel-based GP (GSMGP) models holds great potential [16], it also comes with several challenges. The existing formulation of the GSM kernel is only suitable for handling 1-D data [15], and naively extending it to multidimensional data leads to potential issues, as the number of hyperparameters and the model complexity grow exponentially with the dimensions [1]. In addition, centralized optimization of the GSMGP models in real-world scenarios introduces further challenges. On one hand, the $\mathcal{O}(n^3)$ computational complexity, where n is the training data size, hinders the scalability of GSMGP in big data applications [17], [18], [19]. On the other hand, collecting data centrally becomes difficult in practice, as data are often distributed across locations or devices, and concerns about data privacy often discourage individual data owners from sharing their data [20].

In this article, we address the challenge of constructing an optimal GP kernel for multidimensional and large-scale data. Unlike existing solutions, such as sparse variational GPs with inducing points [21], [22], which remain inefficient due to model complexity and data size, we propose a novel and practical learning framework that leverages advancements in distributed computing to efficiently optimize kernel hyperparameters. Our main contributions are summarized as follows.

- 1) We introduce a novel formulation for the GSM kernel capable of accommodating multidimensional data. Compared with existing formulations [1], [15], our approach effectively reduces the number of hyperparameters while maintaining a good approximation capability. Furthermore, the proposed kernel exhibits a sparsity-promoting property that enables efficient optimization of the hyperparameters.
- 2) We leverage advances in distributed computing and exploit the inherent sparsity of our proposed kernel to develop a practical learning framework for GPs with LMK. The framework, termed sparse linear multiple kernel learning (SLIM-KL), consists of two main components: 1) a quantized alternating direction method of multipliers (ADMMs) scheme, allowing multiple agents to collaboratively learn the kernel hyperparameters while preserving data privacy and reducing communication costs and 2) a distributed successive convex approximation (DSCA) algorithm for solving the local hyperparameter optimization problem in a distributed

manner within the quantized ADMM. As a result, SLIM-KL offers scalability, maintains privacy and ensures efficient communication.

- 3) We provide a comprehensive theoretical analysis of the proposed learning framework. In addition, we conduct extensive experiments on diverse 1-D and multidimensional datasets to evaluate its performance. The results demonstrate that our proposed framework consistently outperforms the existing approaches in terms of prediction performance and computational efficiency.

The remainder of this article is organized as follows. Section II reviews the GP regression (GPR) and GSM kernel. Sections III and IV introduce the proposed kernel formulation and learning framework. Section V presents the theoretical analysis of our proposed framework. Section VI demonstrates the experimental results. Finally, Section VII concludes this article. Key technical proofs and derivations are given in the Appendix.

II. PRELIMINARIES

This section first reviews the salient GPR model. Then, the GSM kernel and the associated hyperparameter optimization problem are introduced.

A. GP Regression

GPR is a powerful, the Bayesian nonparametric approach in machine learning that incorporates prior knowledge and offers predictions with uncertainty estimates [3]. Given an observed dataset $\mathcal{D} \triangleq \{\mathbf{x}_i, y_i\}_{i=1}^n \triangleq \{\mathbf{X}, \mathbf{y}\}$ consisting of n input-output pairs, let us consider the following regression model:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}) + \mathbf{e}, \quad \mathbf{e} \sim \mathcal{N}(\mathbf{0}, \sigma_e^2 \mathbf{I}) \quad (1)$$

where $\mathbf{y}$ is the output, $\mathbf{e}$ is the additive zero-mean Gaussian noise with variance σ_e^2 , and $\mathbf{f}(\mathbf{x})$ is the unknown function we aim to learn. At the heart of GPR lies the assumption that $\mathbf{f}(\mathbf{x})$ follows a GP, defined as a collection of random variables wherein any finite subset follows a joint Gaussian distribution [3]. Mathematically, a GP is expressed as follows:

$$\mathbf{f}(\mathbf{x}) \sim \mathcal{GP}(\mathbf{m}(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta})) \quad (2)$$

where $\mathbf{m}(\mathbf{x})$ is the mean function, typically set to 0 in practice, and $k(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta})$ is the covariance function or kernel, with $\boldsymbol{\theta}$ being its hyperparameters. This kernel function encapsulates our assumptions about the function, such as smoothness and periodicity [3].

In the GPR setting, we are interested in predicting new function values $\mathbf{f}_* \triangleq \mathbf{f}(\mathbf{X}_*)$ for any unseen inputs $\mathbf{X}_*$. Due to the Gaussian nature, the joint distribution of $\mathbf{y}$ and $\mathbf{f}_*$ can be derived as follows:

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K}_{XX} + \sigma_e^2 \mathbf{I}_n & \mathbf{K}_{X_*X}^\top \\ \mathbf{K}_{X_*X} & \mathbf{K}_{X_*X_*} \end{bmatrix}\right). \quad (3)$$

Here, $\mathbf{K}_{XX}$ and $\mathbf{K}_{X_*X_*}$ represent the covariance matrices derived from evaluating the kernel function $k(\mathbf{x}_i, \mathbf{x}_j; \boldsymbol{\theta})$ on the training and test inputs, respectively. Similarly, $\mathbf{K}_{X_*X}$ denotes the cross-covariance matrix evaluated using the training and testing inputs. By applying the results of conditional Gaussian distribution, the predictive distribution $p(\mathbf{f}_* | \mathcal{D}, \mathbf{X}_*) = \mathcal{N}(\mathbf{f}_* | \boldsymbol{\xi}_*, \boldsymbol{\Xi}_*)$ can be computed in closed form, where the

corresponding mean and covariance are, respectively, defined as follows:

$$\xi_* = K_{X_*X} (K_{XX} + \sigma_e^2 I_n)^{-1} y \quad (4a)$$

$$\Xi_* = K_{X_*X_*} - K_{X_*X} (K_{XX} + \sigma_e^2 I_n)^{-1} K_{X_*X}^\top. \quad (4b)$$

This predictive distribution shows that the GPR not only offers a point estimate for the predicted function values (the posterior mean ξ_*) but also quantifies the uncertainty related to these predictions (the posterior covariance Ξ_*).

B. GSM Kernel

Selecting an appropriate kernel is crucial for ensuring good performance of GP models. However, this process is often challenging and traditionally relies on the subjective expert knowledge. Grid SM (GSM) kernel is a universal kernel that eliminates the need for manual kernel design and was originally proposed in [15] to address the model training issues in the SM kernel [14]. Both the SM and GSM kernels leverage the fact that any stationary kernel and its spectral density are Fourier duals [3]. Specifically, the GSM kernel is designed by approximating the spectral density of the underlying kernel function in the frequency domain with a Gaussian mixture, $S(\omega)$, i.e.,

$$S(\omega) = \frac{1}{2} \sum_{q=1}^Q \theta_q [\mathcal{N}(\omega | \mu_q, v_q) + \mathcal{N}(\omega | -\mu_q, v_q)] \quad (5)$$

where Q is the number of Gaussian components and θ_q is the corresponding weight of the q th component in the Gaussian mixture. Unlike the SM kernel, the mean variables $\{\mu_q\}_{q=1}^Q$ and the variance variables $\{v_q\}_{q=1}^Q$ in the GSM kernel are fixed, either uniformly or randomly, to preselected grid points [15]. By taking the inverse Fourier transform of $S(\omega)$, we obtain the formulation for the GSM kernel as follows:

$$k(\tau) = \sum_{q=1}^Q \theta_q \underbrace{\cos(2\pi \tau \mu_q) \exp[-2\pi^2 \tau^2 v_q]}_{k_q(\tau)} \quad (6)$$

where $\tau = |x - x'| \in \mathbb{R}$. In [15], it has been proven that the GSM kernel is able to approximate any stationary kernel arbitrarily well. In addition, the GSM kernel formulated above also demonstrates itself to be an LMK, where $\{k_q(\tau)\}_{q=1}^Q$ are the Q subkernels with weights $\{\theta_q\}_{q=1}^Q$ to be optimized. As an LMK, the GSM subkernels benefit from the low-rank property under reasonable conditions [15]. Consequently, the GSM kernel combines the conciseness of LMKs with the representational power of the SM kernel.

One limitation of the original GSM kernel formulated in (6) is that, it is designed solely for handling 1-D data, i.e., data with an input dimension P equal to 1. Naively extending it to multidimensional data, where $P > 1$, by fixing grid points in $\mathbb{R}^P$, can lead to potential issues [1]. Most notably, the number of grid points grows exponentially with P due to the curse of dimensionality. In other words, substantially more Gaussian components are needed to approximate the spectral density in the P -dimensional case compared to the 1-D case. To illustrate this, consider a 10-D dataset with 100 components per dimension. The total number of components needed would be $Q = 100^{10} = 10^{20}$, which is computationally infeasible. This rapid increase in the number of components makes the

original GSM kernel impractical for high-dimensional data. In Section III, we mitigate this exponential growth of Q by proposing a new formulation for the GSM kernel.

C. Hyperparameter Optimization

In the GPR model, hyperparameter optimization typically involves maximizing the model log marginal likelihood [8], which can be computed analytically due to the Gaussian assumption. Consequently, learning the hyperparameters $\theta = [\theta_1, \dots, \theta_Q]^\top$ in the GSM kernel can be formulated into the following equivalent minimization problem:

$$\hat{\theta} = \arg \min_{\theta} \underbrace{y^\top C^{-1}(\theta) y + \log \det(C(\theta))}_{\triangleq l(\theta)} + \text{constant} \quad (7)$$

where $C(\theta) \triangleq K_{XX} + \sigma_e^2 I_n$. Commonly, gradient-based methods, such as Broyden–Fletcher–Goldfarb–Shanno (BFGS)-Newton and conjugate gradient descent [3], are employed to solve the above minimization problem. However, these methods encounter two primary challenges. First, these methods involve kernel matrix inversion, leading to a prohibitive computational complexity of $\mathcal{O}(n^3)$. Second, the nonconvex nature of the objective often causes these methods to easily get stuck in suboptimal solutions.

Indeed, upon careful examination of the LMK case (e.g., GSM kernel), the optimization problem in (7) is a well-known difference-of-convex programming (DCP) problem [23], where $g(\theta) \triangleq y^\top C^{-1}(\theta) y$ and $h(\theta) \triangleq -\log \det(C(\theta))$ are the convex functions with respect to θ . By exploiting this difference-of-convex structure, the successive convex approximation (SCA) algorithm can be applied to solve the optimization problem efficiently [24]. The basic principle behind the SCA algorithm is to solve the problem in (7) iteratively, where at each iteration a so-called *surrogate function* $\tilde{l}(\theta, \theta^n) : \Theta \times \Theta \mapsto \mathbb{R}$ is minimized, i.e.,

$$\theta^{n+1} = \arg \min_{\theta} \tilde{l}(\theta, \theta^n). \quad (8)$$

Here, $\tilde{l}(\theta, \theta^n)$ is assumed to be: 1) strongly convex on Θ and 2) differentiable with $\nabla_{\theta} \tilde{l}(\theta, \theta^n) = \nabla_{\theta} l(\theta)|_{\theta=\theta^n}$. By fulfilling these conditions, the SCA algorithm is guaranteed to converge to a stationary solution [24]. In our DCP problem, one way to construct such convex surrogate function is to make $h(\theta)$ affine by performing the first-order Taylor expansion

$$\tilde{l}(\theta, \theta^n) = g(\theta) - h(\theta^n) - \nabla_{\theta} h(\theta^n)^\top (\theta - \theta^n). \quad (9)$$

Hence, the convex minimization problem formulated in (8) can be solved efficiently using the commercial solver MOSEK [15], [25], with a computational complexity of $\mathcal{O}(Qn^3)$ per iteration. It is also noteworthy that the constructed surrogate function in (9) is a global upper bound of the objective function $l(\theta)$, so the above SCA algorithm actually coincides with the majorization–minimization (MM) algorithm [26] applied to the original GSM kernel learning [15].

Despite the efficient convex optimization framework offered by SCA, it encounters two primary computational limitations that need to be addressed. First, its centralized computation does not scale well to large or high-dimensional datasets, with the associated computational complexity scaling as $\mathcal{O}(Qn^3)$. Second, SCA requires an access to the full dataset at each

iteration, which is prohibitive for massive distributed datasets prevalent in real-world applications. This also raises privacy concerns, as individual data must be aggregated to a central unit for processing.

III. GSMP KERNEL FOR MULTIDIMENSIONAL DATA

The original GSM kernel, introduced in Section II-B, is primarily designed for 1-D input data ($P = 1$), limiting its utility for multidimensional data. When applied to multidimensional data, the GSM kernel experiences an exponential increase in both model and computational complexity due to the rapid growth in the number of candidate grid points [1]. This exponential growth poses computational challenges and complicates the application of the GSM kernel in high-dimensional problems.

A. Proposed GSMP Kernel

To address the limitations of the GSM kernel, we propose a new formulation called the GSMP kernel. The GSMP kernel is formulated by taking the product of the 1-D subkernels, $k_q(\tau^{(p)})$, across all dimensions

$$k(\tau) = \sum_{q=1}^Q \theta_q \prod_{p=1}^P \underbrace{\exp\{-2\pi^2 \tau^{(p)^2} v_q^{(p)}\}}_{\text{1-D subkernel: } k_q(\tau^{(p)})} \cos(2\pi \tau^{(p)} \mu_q^{(p)}) \quad (10)$$

where $v_q^{(p)}$ and $\mu_q^{(p)}$ are fixed for dimension p , $\forall p \in \{1, 2, \dots, P\}$, as in the original 1-D GSM kernel, see (6); and $\{\theta_q\}_{q=1}^Q$ are the weights to be optimized. The GSMP kernel formulated in (10) is essentially an LMK, preserving the favorable difference-of-convex structure for hyperparameter optimization.

To construct the GSMP kernel, we need to generate fixed grid points for each subkernel component $k_q(\tau^{(p)})$ along each dimension p , $p \in \{1, 2, \dots, P\}$. We first fix the variance variables $v_q^{(p)}$, $\forall q, p$ to a small constant, e.g., 0.001. Then, we empirically sample Q frequencies, either uniformly or randomly, from the frequency range $[0, \mu_u^{(p)}]$ to obtain $[\mu_1^{(p)}, \dots, \mu_Q^{(p)}]$, where $\mu_u^{(p)}$ is the highest frequency set to be equal to $1/2$ over the minimum input spacing between the two adjacent training data points in the p th dimension. Using the sampled frequencies and fixed variance, we can evaluate the GSMP kernel in (10), leaving the weights $\{\theta_q\}_{q=1}^Q$ to be tuned. In our experiments, without any further specification, we set the number of components Q to scale linearly with the number of dimensions. This allows the GSMP kernel to effectively alleviate the curse of dimensionality while ensuring the stability of the optimization process.

By applying a Fourier transform on the GSMP kernel, we can obtain the corresponding spectral density of the GSMP kernel, which appears as a Gaussian mixture

Theorem 1: The spectral density of the GSMP kernel, defined in (10), is a Gaussian mixture given by

$$S(\omega) = \frac{1}{2^P} \sum_{q=1}^Q \theta_q \prod_{p=1}^P [\mathcal{N}(\omega^{(p)} | \mu_q^{(p)}, v_q^{(p)}) + \mathcal{N}(\omega^{(p)} | -\mu_q^{(p)}, v_q^{(p)})]. \quad (11)$$

Proof: The proof can be found in Appendix A. $\square$

It is worth noting that Gaussian mixtures are dense in the set of all distributions [27]. Hence, the Fourier dual of this set, which corresponds to the set of stationary kernels, is also dense [14]. That is to say, Theorem 1 implies that, given large enough components Q , the GSMP kernel has the ability to approximate the underlying stationary kernel well [15].

B. Properties of GSMP Kernel

The GSMP kernel exhibits several advantageous properties that enhance its utility over existing formulations like the GSM kernel. One good property of the GSMP kernel is that it requires significantly fewer grid points to achieve a good approximation capability compared with the multidimensional GSM kernel proposed in [1]. We illustrate this property with a simple example.

Example 1: In the 2-D space, according to (11), the spectral density of the GSM kernel is given by [1, eq. (11)]

$$\begin{aligned} S_{\text{GSM}}(\omega) &= \sum_{q=1}^Q \frac{\theta_q}{2} [\mathcal{N}(\omega^{(1)} | \mu_q^{(1)}, v_q^{(1)}) \mathcal{N}(\omega^{(2)} | \mu_q^{(2)}, v_q^{(2)}) \\ &\quad + \mathcal{N}(\omega^{(1)} | -\mu_q^{(1)}, v_q^{(1)}) \mathcal{N}(\omega^{(2)} | -\mu_q^{(2)}, v_q^{(2)})] \end{aligned} \quad (12)$$

while the spectral density of the GSMP kernel is

$$\begin{aligned} S_{\text{GSMP}}(\omega) &= \sum_{q=1}^Q \frac{\theta_q}{4} [\mathcal{N}(\omega^{(1)} | \mu_q^{(1)}, v_q^{(1)}) \mathcal{N}(\omega^{(2)} | \mu_q^{(2)}, v_q^{(2)}) \\ &\quad + \mathcal{N}(\omega^{(1)} | -\mu_q^{(1)}, v_q^{(1)}) \mathcal{N}(\omega^{(2)} | -\mu_q^{(2)}, v_q^{(2)}) \\ &\quad + \mathcal{N}(\omega^{(1)} | \mu_q^{(1)}, v_q^{(1)}) \mathcal{N}(\omega^{(2)} | -\mu_q^{(2)}, v_q^{(2)}) \\ &\quad + \mathcal{N}(\omega^{(1)} | -\mu_q^{(1)}, v_q^{(1)}) \mathcal{N}(\omega^{(2)} | \mu_q^{(2)}, v_q^{(2)})]. \end{aligned} \quad (13)$$

The GSMP density contains the $2Q$ densities of the GSM plus additional $2Q$ Gaussian densities, for approximating the underlying density, as demonstrated in (12) and (13).

As illustrated in Example 1, the spectral density of the GSMP kernel exhibits a greater number of Gaussian densities for approximating the spectral density of the underlying kernel, compared to that of the GSM kernel. Extending this to a P -dimensional space, the number of additional Gaussian densities is $Q \times (2^P - 2)$ for the same number of grid points, Q , and dimensionality, P . Consequently, the GSM kernel requires an additional $Q \times (2^{P-1} - 1)$ grid points or subkernels to match the GSMP kernel's capacity. This structural advantage of the GSMP kernel, facilitating fewer preselected grid points or subkernels, not only alleviates the curse of dimensionality but also reduces both the model and computational complexity. Further results and discussions are deferred to Section VI-A.

Another property of the GSMP kernel is that, the solution to the corresponding hyperparameter optimization problem formulated in (7) is sparse, as succinctly summarized in the following theorem.

Theorem 2: Every local minimum of the hyperparameter optimization problem formulated in (7) for the proposed GSMP kernel, is achieved at a sparse solution, regardless of whether noise is present or not.

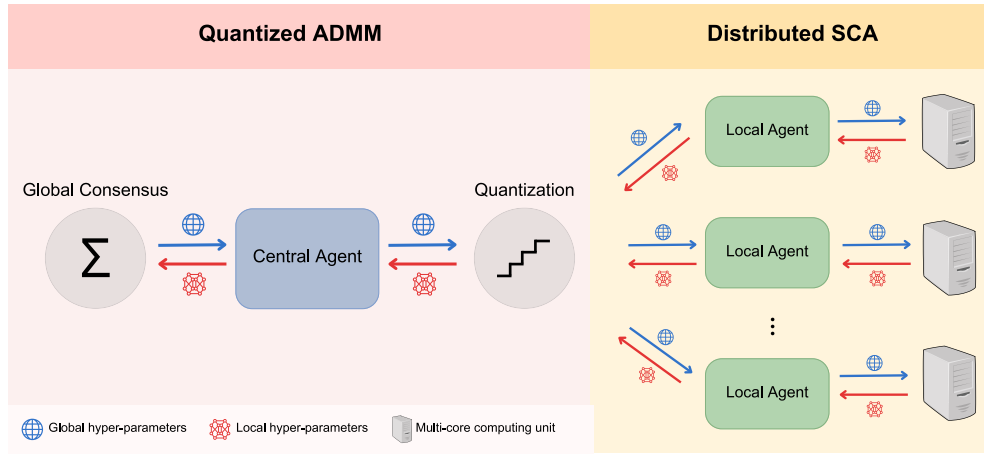


Fig. 1. Proposed SLIM-KL framework, featuring a quantized ADMM scheme for collaborative hyperparameter learning across multiple agents, and a DSCA algorithm for local optimization using multicore computing units.

Proof: The proof can be found in Appendix B. $\square$

The solution's sparsity is a notable advantage for the proposed GSMP kernel. First, it ensures that only the grid points or subkernels deemed significant by the data are pinpointed, enhancing the kernel's interpretability. Second, by refraining from utilizing all grid points for data fitting, the issue of over-parameterization can be effectively alleviated. In Section IV, we demonstrate how to exploit this inherent sparsity to design an efficient optimization framework for learning the hyperparameters.

IV. SPARSITY-AWARE DISTRIBUTED LEARNING FRAMEWORK

Previously, the SCA algorithm introduced in Section II-C suffers from scalability issues as solving the optimization problem in (8) becomes computationally prohibitive for big data, i.e., with large n . Moreover, large amounts of labeled training data are usually aggregated from a large number of local agents or mobile devices, which may cause severe data privacy issues [20]. Practical constraints faced by agents, such as limited bandwidth, battery power, and computing resources [28], further exacerbate these issues. For instance, the extensive information exchange between diverse local agents often exceeds the available bandwidth for data transmission, as discussed in [29].

To address these challenges, we propose an efficient, sparsity-aware learning framework, termed SLIM-KL, as depicted in Fig. 1, for learning the proposed kernel. Unlike the doubly DSCA framework proposed in [1], SLIM-KL integrates a quantized ADMMs scheme to enable collaborative learning with reduced communication costs and practical consideration of transmission constraints. This quantization scheme is a key differentiator, ensuring efficient transmission of model hyperparameters between distributed agents. In addition, SLIM-KL employs a DSCA algorithm for scalable training of a large number of hyperparameters while leveraging the sparsity-promoting property of the proposed GSMP kernel. Note that while this article focuses on the GSMP kernel, the proposed learning framework can be applied to any LMKs. The key notations used in our SLIM-KL framework are summarized in the Nomenclature. In Sections IV-A and IV-B, we detail the two key components of our SLIM-KL framework.

A. Quantized ADMM

ADMM has emerged as a promising algorithm for developing a principled distributed learning framework [30], enabling multiple agents to solve large-scale distributed GPR problems [10]. In this work, we consider a variant of ADMM that incorporates quantization [31], as illustrated in Fig. 1 (quantized ADMM). Quantization is a widely used technique in wireless systems to compress information by reducing transmission bits [32], [33]. Mathematically, consider $N \in \mathbb{N}$ multicore agents cooperatively learning a global GPR model with the proposed GSMP kernel, the learning problem can be formulated as follows:

$$\begin{aligned} \min \quad & \sum_{j=1}^N l(\xi_j; \mathcal{D}_j) \\ \text{s. t.} \quad & \xi_j - \theta = \mathbf{0}, \quad j \in \{1, \dots, N\} \\ & \theta \in \Theta, \quad \xi_j \in \Theta, \quad j \in \{1, \dots, N\} \end{aligned} \quad (14)$$

where $\mathcal{D}_j$ denotes the local dataset owned by the j th agent with $|\mathcal{D}_j| = n/N$, ξ_j denotes the local hyperparameters, and θ denotes the global hyperparameters (the weights of GSMP kernel). In this setup, we assume that each agent has the ability to store and perform computations with infinite precision. However, when it comes to transmission, all agents are constrained to sending quantized data that can be received without any error.

Let θ^t be the hyperparameters at iteration t . Suppose for each element $i \in \{1, \dots, Q\}$ of θ^t , it holds that $\theta_i^t \in [\theta_{\min}^t, \theta_{\max}^t]$, where $\theta_{\min}^t$ and $\theta_{\max}^t$ denote the smallest and largest entries of θ^t , respectively. We aim to obtain a quantized value of θ_i^t with length l_t bits. We define $L_t = 2^{l_t}$ quantization points given by the set $\Lambda = \{\tau_1, \dots, \tau_{L_t}\}$, where $\tau_1 = \theta_{\min}^t$ and $\tau_{L_t} = \theta_{\max}^t$. The points in this set are uniformly spaced such that $\Delta = \tau_{j+1} - \tau_j$ for $j \in \{1, \dots, L_t - 1\}$ [34]. It follows that $\Delta = (\theta_{\max}^t - \theta_{\min}^t) / (L_t - 1)$. Here, $\Delta > 0$ denotes the quantization resolution, where smaller Δ allows more precise transmissions but requires more bits and vice versa. Then, for $\theta_i^t \in [\tau_j, \tau_{j+1})$, we can quantize θ_i^t using the following stochastic quantization function [35]:

$$\mathcal{Q}(\theta_i^t) = \begin{cases} \tau_j, & \text{with probability } 1 - (\theta_i^t - \tau_j) / \Delta \\ \tau_{j+1}, & \text{with probability } (\theta_i^t - \tau_j) / \Delta. \end{cases} \quad (15)$$

In Section V, we will establish that quantized ADMM is guaranteed to converge to a stationary point, thanks to the properties of the stochastic quantization used.

To solve the optimization problem in (14), we first formulate the augmented Lagrangian as follows:

$$\begin{aligned} \mathcal{L}(\xi_1, \dots, \xi_N, \lambda_1, \dots, \lambda_N, \theta) \\ \triangleq \sum_{j=1}^N \left(l(\xi_j; \mathcal{D}_j) + \lambda_j^\top (\xi_j - \theta) + \frac{\rho_j}{2} \|\xi_j - \theta\|_2^2 \right) \end{aligned} \quad (16)$$

where λ_j is the dual variable and ρ_j is a preselected penalty parameter.

By imposing a quantization operation on θ and ξ_j immediately after the θ -update and ξ_j -update, the sequential update in quantized ADMM for the $(t+1)$ th iteration can be decomposed as follows:

$$\theta^{t+1} = \frac{1}{N} \sum_{j=1}^N \left(\xi_{j,[Q]}^t + \frac{1}{\rho_j} \lambda_j^t \right) \quad (17a)$$

$$\theta_{[Q]}^{t+1} = \mathcal{Q}(\theta^{t+1}) \quad (17b)$$

$$\begin{aligned} \xi_j^{t+1} = \arg \min_{\xi_j} l(\xi_j; \mathcal{D}_j) + (\lambda_j^t)^\top (\xi_j - \theta_{[Q]}^{t+1}) \\ + \frac{\rho_j}{2} \|\xi_j - \theta_{[Q]}^{t+1}\|_2^2, \quad j \in \{1, \dots, N\} \end{aligned} \quad (17c)$$

$$\xi_{j,[Q]}^{t+1} = \mathcal{Q}(\xi_j^{t+1}) \quad (17d)$$

$$\lambda_j^{t+1} = \lambda_j^t + \rho_j (\xi_{j,[Q]}^{t+1} - \theta_{[Q]}^{t+1}). \quad (17e)$$

Equation (17a) is the global consensus step, averaging the local agents' updates to form the new global hyperparameter θ^{t+1} . Equation (17c) is the local update step, where each agent j optimizes its local hyperparameters ξ_j to align with the global hyperparameter using its local data $\mathcal{D}_j$. Equation (17e) updates the dual variables λ_j to maintain consistency between the global and local hyperparameters. Equations (17b) and (17d) denote the global and local quantization steps, respectively. This quantized ADMM scheme allows multiple agents to collaboratively learn the hyperparameters while preserving data privacy and reducing communication costs. Another useful property of the quantized ADMM is that if one local agent gets stuck at a poor local minimum of the GSMP kernel hyperparameter, the consensus step may help the agent resume from a more reasonable point in the next iteration.

B. DSCA Algorithm

It is worth noting that the local minimization problem formulated in (17c) is not only nonconvex but also nonseparable, rendering distributed computational infeasible. Hence, solving it with respect to the entire ξ_j can be computationally demanding for large Q . To alleviate this computational burden, we propose the DSCA algorithm to optimize the hyperparameters in a distributed manner, as depicted in Fig. 1 (DSCA). The key idea is to split the hyperparameters ξ_j into s blocks and optimize them in parallel across s computing units, $s \in \mathbb{N}$. Concretely, by exploiting the Cartesian product structure of the feasible set Θ in the GSMP kernel, i.e., $\Theta = \Theta_1 \times \Theta_2 \times \dots \times \Theta_s$ with $\Theta_i \subseteq \mathbb{R}^{Q/s}$, we first partition the optimization variable into s blocks, $\xi_j =$

$[\xi_{j,1}, \xi_{j,2}, \dots, \xi_{j,s}]^\top$. Then, we can construct the associated surrogate function for $l(\xi_j; \mathcal{D}_j)$ in (17c), that is additively separable in the blocks

$$\tilde{l}(\xi_j, \xi_j^\eta) = \sum_{i=1}^s \tilde{l}_i(\xi_{j,i}, \xi_j^\eta). \quad (18)$$

Assumption 1: Each surrogate function $\tilde{l}_i(\xi_{j,i}, \xi_j^\eta) : \Theta_i \times \Theta \mapsto \mathbb{R}, i \in \{1, \dots, s\}$, satisfies the following conditions.

- 1) $\tilde{l}_i(\xi_{j,i}, \xi_j^\eta)$ is strongly convex on Θ_i .
- 2) $\tilde{l}_i(\xi_{j,i}, \xi_j^\eta)$ is differentiable with

$$\nabla_{\xi_{j,i}} \tilde{l}_i(\xi_{j,i}, \xi_j^\eta) = \nabla_{\xi_{j,i}} l(\xi_j) |_{\xi_j = \xi_j^\eta}. \quad (19)$$

To fulfill the two conditions given in Assumption 1, we employ a similar strategy as the vanilla SCA and construct $\tilde{l}_i(\xi_{j,i}, \xi_j^\eta)$ in (18) as follows:

$$\begin{aligned} \tilde{l}_i(\xi_{j,i}, \xi_j^\eta) = g(\xi_{j,i}, \xi_{j,-i}^\eta) - h(\xi_j^\eta) \\ - \nabla_{\xi_{j,i}} h(\xi_j^\eta)^\top (\xi_{j,i} - \xi_{j,i}^\eta) \end{aligned} \quad (20)$$

where

$$\nabla_{\xi_{j,i}} h(\xi_j^\eta) = \left[\begin{array}{c} -\text{Tr} \left(C^{-1}(\xi_j) \frac{\partial C(\xi_j)}{\partial \xi_{j,1+(i-1)Q/s}} \right) \\ -\text{Tr} \left(C^{-1}(\xi_j) \frac{\partial C(\xi_j)}{\partial \xi_{j,2+(i-1)Q/s}} \right) \\ \vdots \\ -\text{Tr} \left(C^{-1}(\xi_j) \frac{\partial C(\xi_j)}{\partial \xi_{j,iQ/s}} \right) \end{array} \right] \Big|_{\xi_j = \xi_j^\eta}$$

and $\xi_{j,-i}^\eta = [\xi_{j,1}^\eta, \dots, \xi_{j,i-1}^\eta, \xi_{j,i+1}^\eta, \dots, \xi_{j,s}^\eta]^\top$. The DSCA algorithm solves the following convex optimization problem in parallel for all $i \in \{1, \dots, s\}$:

$$\xi_{j,i}^{\eta+1} = \arg \min_{\xi_{j,i}} \ell(\xi_{j,i}, \xi_j^\eta) \quad (21)$$

where $\ell(\xi_{j,i}, \xi_j^\eta) \triangleq \tilde{l}_i(\xi_{j,i}, \xi_j^\eta) + (\lambda_{j,i}^t)^\top (\xi_{j,i} - \theta_i^{t+1}) + (\rho_j/2) \|\xi_{j,i} - \theta_i^{t+1}\|_2^2$. However, it has been observed that directly using CVX [36], a package designed for constructing and solving convex programs, to solve (21) is computationally demanding. Since the term $\tilde{l}_i(\xi_{j,i}, \xi_j^\eta)$ in (21) contains a matrix fractional component, we can reformulate the problem into a conic form, by introducing auxiliary variables z , w , and v

$$\begin{aligned} \min_{\xi_{j,i}, z, w, v} \quad & (\mathbf{1}^\top z) + \left(\lambda_{j,i}^t - \nabla_{\xi_{j,i}} h(\xi_j^\eta) \right)^\top \xi_{j,i} + \frac{\rho_j}{2} v \\ \text{s.t.} \quad & \left\| \begin{bmatrix} 2w_0 \\ z_0 - 1 \end{bmatrix} \right\|_2 \leq z_0 + 1 \\ & \left\| \begin{bmatrix} 2w_k \\ z_k - \xi_{j,k+(i-1)Q/s} \end{bmatrix} \right\|_2 \leq z_k + \xi_{j,k+(i-1)Q/s} \\ & k \in \{1, 2, \dots, Q/s\} \\ & y = \sum_{k=1}^{Q/s} L_{k+(i-1)Q/s} w_k \end{aligned}$$

$$\begin{aligned}
& + \left(\sum_{n \neq i} \sum_{k=1}^{Q/s} \zeta_{j,k+(n-1)Q/s}^\eta \mathbf{K}_{k+(n-1)Q/s} + \sigma_e^2 \mathbf{I}_n \right)^{\frac{1}{2}} \mathbf{w}_0 \\
2v \frac{1}{2} & \geq \sum_{k=1}^{Q/s} \left(\zeta_{j,k+(i-1)Q/s} - \theta_{k+(i-1)Q/s}^{t+1} \right)^2 \\
\zeta_{j,i} & \geq \mathbf{0}, \quad \mathbf{z} \geq \mathbf{0}, \quad v \geq 0.
\end{aligned} \tag{22}$$

We provide the detailed steps for this reformulation in Appendix C. The problem presented in (22) is now in conic form and can be solved more efficiently using MOSEK [15], [25]. After the local agents have optimized their local hyperparameters using DSCA in the t th iteration, the ADMM consensus step formulated in (17a) is conducted to reach a global consensus. At this point, we have introduced the SLIM-KL framework. The pseudocode for implementing SLIM-KL is summarized in Algorithm 1.

Algorithm 1 SLIM-KL for LMK Learning

```

1 Initialization:  $t = 0$ ,  $\{\rho_j, \zeta_j^0, \lambda_j^0\}_{j=1}^N$ ,  $N$  local datasets  $\{\mathcal{D}_j\}_{j=1}^N$ 
   in  $N$  local computing agents, quantization function  $\mathcal{Q}(\cdot)$ ;
2 while Iteration stopping criteria are not satisfied do
3   Obtain the global hyper-parameters  $\theta^{t+1}$  by Eq. (17a);
4   Apply quantization on the global hyper-parameters using
   Eq. (17b) and obtain  $\theta_{[Q]}^{t+1}$ ;
5   for  $j \in [1, N]$  do
6     Obtain the local hyper-parameters  $\zeta_j^{t+1}$  by utilizing
     DSCA to solve Eq. (22) in parallel using  $s$  units;
7     Apply quantization on the local hyper-parameters using
     Eq. (17d) and obtain  $\zeta_{j,[Q]}^{t+1}$ ;
8   end
9   Obtain the dual variables by Eq. (17e);
10 end
11 return  $\theta^t$ 

```

Remark 1: By utilizing N local agents and s computing units, the overall computational complexity of SLIM-KL at each iteration is reduced from $\mathcal{O}(Qn^3)$ to $\mathcal{O}(Qn^3/sN^3)$. That is, as the number of agents N and computing units s increase, the computational burden of optimizing the hyperparameters can be significantly reduced, especially when dealing with a large number of hyperparameters Q and training samples n .

Remark 2: The design of SLIM-KL matches perfectly with the sparse nature of the proposed GSMP kernel, see Theorem 2. Many weights/hyperparameters of the subkernels in the GSMP kernel are inherently zero or near-zero valued, so quantization can enhance this sparsity by rounding even more of these weights to 0. This enhanced sparsity could aid convergence and guide the algorithm toward a favorable local minimum, as noted in [31].

V. ANALYSIS

In this section, we present the theoretical analysis of the SLIM-KL framework proposed in Section IV. First, we show that the DSCA algorithm solving the local minimization problem within quantized ADMM is guaranteed to converge, which is succinctly summarized as follows.

Theorem 3: By employing DSCA, the objective of the local minimization problem, see (17c), is nonincreasing, and the

sequence of feasible points $\{\zeta_j^\eta\}_{\eta \in \mathbb{N}}$ satisfies

$$\lim_{\eta \rightarrow \infty} \|\zeta_j^{\eta+1} - \zeta_j^\eta\| = 0. \tag{23}$$

Proof: The proof can be found in Appendix D. $\square$

Theorem 3 suggests that within the iteration of quantized ADMM, the local minimization problem is solved exactly [37]. Building upon the result from Theorem 3, we can show that the quantized ADMM-based SLIM-KL framework is guaranteed to reach a stationary point in a mean sense, according to the following theorem.

Theorem 4: The SLIM-KL framework ensures that, under the stochastic quantization scheme, the sequence $\{\zeta_j^t\}_{j=1}^N$ converges to θ^t in the mean sense as $t \rightarrow \infty$, with the variance of ζ_j^t stabilizing at a finite value which depends on the quantization resolution Δ .

Proof: We first refer to the following lemma.

Lemma 1: With stochastic quantization, each $\theta \in \mathbb{R}$ can be unbiasedly estimated as follows:

$$\mathbb{E}[\mathcal{Q}(\theta)] = \theta \tag{24}$$

and the associated quantization error is bounded by

$$\mathbb{E}[(\theta - \mathcal{Q}(\theta))^2] \leq \frac{\Delta^2}{4}. \tag{25}$$

Proof: The proof can be found in Appendix E. $\square$

It follows that, by taking expectation on both sides of (17a) and (17e), we can get

$$\mathbb{E}[\theta^{t+1}] = \frac{1}{N} \sum_{j=1}^N \left(\mathbb{E}[\zeta_{j,[Q]}^t] + \frac{1}{\rho_j} \mathbb{E}[\lambda_j^t] \right) \tag{26a}$$

$$\mathbb{E}[\lambda_j^{t+1}] = \mathbb{E}[\lambda_j^t] + \rho_j \left(\mathbb{E}[\zeta_{j,[Q]}^{t+1}] - \mathbb{E}[\theta_{[Q]}^{t+1}] \right). \tag{26b}$$

Note that Lemma 1 implies that $\mathbb{E}[\zeta_{j,[Q]}^{t+1}] = \zeta_j^{t+1}$ and $\mathbb{E}[\theta_{[Q]}^{t+1}] = \theta^{t+1}$. Since $\mathbb{E}[\lambda_j^0] = \lambda_j^0$, it is easy to see that (26a) and (26b) takes exactly the same iterations in the mean sense as the consensus ADMM, i.e., ADMM without quantization scheme. Therefore, the convergence in the mean of $\{\zeta_j^t\}_{j=1}^N$ to θ^t is guaranteed due to the convergence of consensus ADMM established in [30], [38], and [31]. Moreover, let $\epsilon^t = \zeta_j^t - \mathcal{Q}(\zeta_j^t)$ be the quantization error of ζ_j^t . By Lemma 1, we can show that it satisfies: $\mathbb{E}[\epsilon^t] = 0$ and $\text{Var}(\epsilon^t) \leq \Delta^2/4$. That is, the quantization error ϵ^t introduced at each iteration has an expected value of 0 and a variance that is bounded by $\Delta^2/4$. As the quantization resolution Δ decreases, the variance of the quantization error also decreases, enhancing the convergence rate of the algorithm. Conversely, larger Δ increases the variance bound, potentially slowing convergence. Thus, as $t \rightarrow \infty$, the variance of ζ_j^t stabilizes to a finite value that depends on the quantization resolution Δ [31]. This completes the proof for Theorem 4. $\square$

VI. EXPERIMENTS

In this section, we present experimental validations for the kernel formulation and learning framework proposed in Sections III and IV. The efficiency and approximation capability of the proposed GSMP kernel are demonstrated in Section VI-A, followed by a comparative analysis between our method and some competing approaches in

TABLE I
PREDICTION MSE OF THE GP WITH GSMP KERNEL VERSUS THE GP
WITH GSM KERNEL [1]

Configuration	1	2	3	4	5
GSM	0.0172	0.5564	0.0046	0.9431	7.2646E-03
GSMP	0.0101	0.1732	0.0063	0.3769	8.3232E-03

Section VI-B. Sections VI-C and VI-D highlight the scalability and communication efficiency of our learning framework. Finally, Section VI-E demonstrates the performance of our method on real-world long time-series forecasting tasks. Our implementation code is accessible at: <https://github.com/richardcsuwandi/SLIM-KL>.

A. Approximation Capability of GSMP Kernel

This section demonstrates the approximating capability of the GSMP kernel discussed in Section III-B. Specifically, we empirically validate the capability of the GSMP kernel to maintain a good approximation while reducing the number of hyperparameters, as highlighted in Example 1. We generate synthetic datasets with known underlying spectral densities and compare the performance of the GSMP kernel against the original GSM kernel proposed in [1]. The generated datasets consist of 500 samples from a GP with the SM kernel. The GP inputs are uniformly distributed within the domain $[0, 10]^2$, with 480 samples used for training and 20 samples reserved for testing. We compare the GP performance with the GSM and GSMP kernels under different configurations as outlined as follows.

- 1) *Configuration 1*: The SM kernel has an underlying four-mode Gaussian mixture spectral density, $S(\omega) = \sum_{i=1}^4 \alpha_i \mathcal{N}(\mu_i, V_i)$, where $\alpha_1 = \dots = \alpha_4 = 5$, $\mu_1 = [-2, -2]^\top$, $\mu_2 = [-2, 2]^\top$, $\mu_3 = [2, -2]^\top$, $\mu_4 = [2, 2]^\top$, and $V_1 = \dots = V_4 = 0.1I$.
- 2) *Configuration 2*: The SM kernel has an underlying four-mode Gaussian mixture spectral density, $S(\omega) = \sum_{i=1}^4 \alpha_i \mathcal{N}(\mu_i, V_i)$, where $\alpha_1 = \dots = \alpha_4 = 5$, $\mu_1 = [-2, -2]^\top$, $\mu_2 = [-3, 3]^\top$, $\mu_3 = [3, -3]^\top$, $\mu_4 = [2, 2]^\top$, and $V_1 = \dots = V_4 = 0.1I$.
- 3) *Configuration 3*: The SM kernel has an underlying two-mode Gaussian mixture spectral density, $S(\omega) = \sum_{i=1}^2 \alpha_i \mathcal{N}(\mu_i, V_i)$, where $\alpha_1 = \alpha_2 = 5$, $\mu_1 = [-2, -2]^\top$, $\mu_2 = [2, 2]^\top$, and $V_1 = V_2 = 0.1I$.
- 4) *Configuration 4*: The SM kernel has an underlying four-mode Gaussian mixture spectral density, $S(\omega) = \sum_{i=1}^4 \alpha_i \mathcal{N}(\mu_i, V_i)$, where $\alpha_1 = \alpha_4 = 15$, $\alpha_2 = \alpha_3 = 5$, $\mu_1 = [-2, -2]^\top$, $\mu_2 = [-3, 3]^\top$, $\mu_3 = [3, -3]^\top$, $\mu_4 = [2, 2]^\top$, and $V_1 = \dots = V_4 = 0.1I$.
- 5) *Configuration 5*: The SM kernel has an underlying six-mode Gaussian mixture spectral density, $S(\omega) = \sum_{i=1}^6 \alpha_i \mathcal{N}(\mu_i, V_i)$, where $\alpha_1 = \alpha_6 = 10$, $\alpha_2 = \alpha_5 = 10$, $\alpha_3 = \alpha_4 = 5$, $\mu_1 = [0.1, 0.1]^\top$, $\mu_2 = [1, 1]^\top$, $\mu_3 = [1.5, 1.5]^\top$, $\mu_4 = [-1.5, -1.5]^\top$, $\mu_5 = [-1, -1]^\top$, $\mu_6 = [-0.1, -0.1]^\top$, and $V_1 = \dots = V_6 = 0.05I$.

For the comparison, we set $Q = 50$ and sample the grid points from only the positive quadrant, that is, $\mu_q^{(p)}$ are uniformly sampled from $[0, 3]$ and the variance variables $v_q^{(p)}$ are set to 0.01. The spectral density results of the learned kernels are depicted in Fig. 2, and the GP prediction results are presented in Table I.

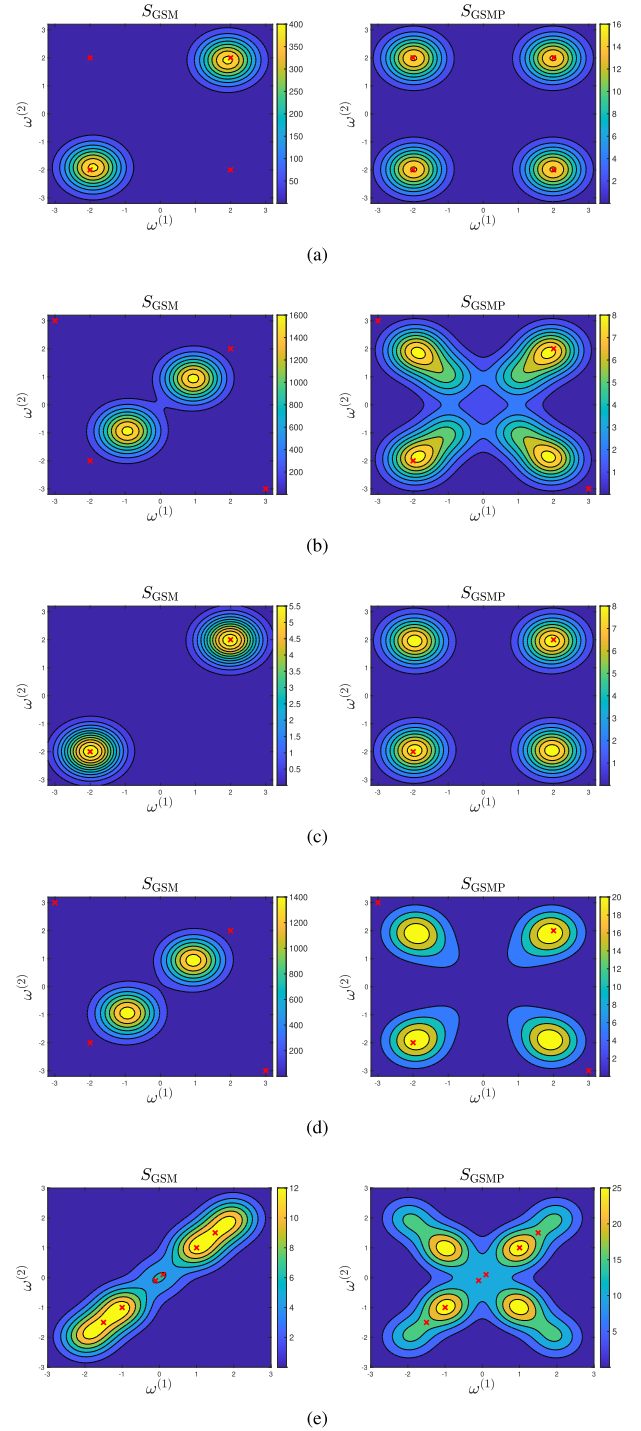


Fig. 2. Learned spectral density of multidimensional GSM kernel (left) versus the learned spectral density of GSMP kernel (right). The cross symbols represent the modes of the ground truth. (a) Configuration 1. (b) Configuration 2. (c) Configuration 3. (d) Configuration 4. (e) Configuration 5.

From the results of configurations 1, 2, and 4 in Fig. 2, we observe that when the density modes of the underlying kernel lie along the main diagonal, the GSM kernel struggles to represent this density accurately. This is because the GSM kernel's fixed grid points support the density on the antidiagonal but not the density on the main diagonal. In contrast, the GSMP kernel contains additional $2Q$ Gaussian densities, enabling it to capture the main diagonal's density effectively. This superior fitting of the underlying

TABLE II
DETAILS OF THE SELECTED REAL DATASETS

Dataset	ECG	CO2	Unemployment	ALE	CCCP	Toxicity	Concrete	Wine	Water
Training Size	680	481	380	80	1000	436	824	1279	5000
Test Size	20	20	20	20	250	110	206	320	100
Dimension	1	1	1	4	4	8	8	11	11

TABLE III

PERFORMANCE COMPARISON BETWEEN THE PROPOSED GSMPPG WITH SLIM-KL ($s = 4$, $N = 2$, AND $\Delta = 0.01$) AND ITS COMPETITORS IN TERMS OF THE PREDICTION MSE. THE LOWEST MSE VALUE IS HIGHLIGHTED IN BOLD. THE PERCENTAGE IN THE BRACKETS INDICATES THE SPARSITY LEVEL OF THE SOLUTION. THE LOWER THE VALUE, THE SPARSER THE SOLUTION

Dataset	GSMPPG	GSMGP	SMGP	SSGP	SEGP	LSTM	Informer
ECG	1.2E-02 (0.8%)	1.2E-02 (0.8%)	1.9E-02 (100%)	1.6E-01	1.6E-01	1.6E-01	5.4E-02
CO2	3.7E-01 (0.6%)	6.2E-01 (1.0%)	7.4E-01 (100%)	2.0E+02	1.5E+03	2.9E+02	8.4E+01
Unemployment	2.0E+03 (2.8%)	2.2E+03 (4.6%)	7.7E+03 (100%)	2.1E+04	5.6E+05	1.7E+05	3.8E+03
ALE	2.5E-02 (0.25%)	1.9E+00 (0.5%)	3.7E-01 (100%)	5.8E-02	3.7E-02	3.4E-02	1.8E-01
CCCP	1.6E+01 (0.25%)	1.9E+02 (0.25%)	2.1E+05 (100%)	1.6E+01	1.7E+01	2.8E+02	1.4E+05
Toxicity	1.4E+00 (1.13%)	2.7E+02 (1.13%)	5.3E+00 (100%)	1.7E+00	1.4E+00	1.7E+00	2.9E+00
Concrete	5.9E+01 (0.13%)	5.6E+02 (0.25%)	1.7E+03 (100%)	3.5E+02	1.3E+02	1.4E+02	2.0E+02
Wine	4.6E-01 (0.09%)	7.4E+03 (0.09%)	3.1E+01 (100%)	5.6E-01	4.4E+00	4.7E-01	1.6E+00
Water	1.6E-04 (1.27%)	1.7E-04 (1.0%)	5.5E-03 (100%)	2.7E-04	1.6E-04	4.2E-04	9.2E-02

density translates into GSMP kernel's enhanced prediction performance, as corroborated by the results shown in Table I. If one wants to compensate for the performance deficiencies of the GSM kernel, it requires fixing additional Q grid points, totaling $2Q$ grid points. Conversely, the GSMP kernel achieves comparable performance while only requiring Q grid points. In configurations 3 and 5 of Fig. 2, where the density pattern aligns with the antidiagonal, the GSM kernel approximates this density adequately. On the other hand, the GSMP kernel not only captures the underlying antidiagonal density but also introduces additional density on the main diagonal. However, the GP prediction performance using the GSMP kernel, as showcased in Table I, demonstrates a marginal difference from the performance achieved with the GSM kernel. This indicates that the GSMP kernel, with its additional Gaussian densities, still achieves comparable performance without the need for extra grid points, thus maintaining a lower computational complexity.

The above empirical findings highlight that the GSMP kernel not only effectively approximates the underlying kernel, thus ensuring good GP prediction performance, but also notably diminishes the number of hyperparameters. This reduction significantly mitigates both model and computational complexities, supporting our claim in Section III-B.

B. Prediction Performance

This section showcases the prediction performance of the proposed GSMP kernel-based GP (GSMPPG) trained using SLIM-KL on nine real-world datasets. The dimensionality of these datasets varies from 1 to 11, with more details provided in Table II. The setup for training the GSMPPG is listed as follows. The number of mixture components Q is set to 500 for the 1-D case and $100 \times P$ for the multidimensional case. The mean variables $\mu_q^{(p)}$ are uniformly sampled from the frequency range mentioned in Section III-A, and the variance variables $v_q^{(p)}$ are fixed to a small constant, 0.001. The weights θ and the dual variables $\{\lambda_j\}_{j=1}^N$ are initialized to zeros. The penalty parameter $\{\rho_j\}_{j=1}^N$ is initialized to 10^{-10} and adaptively

updated using the residual balancing strategy [39]. The noise variance parameter σ_e^2 is estimated using a cross-validation filter type method [40].

For comparison, we consider six other models, the original GSMGP [1], an SM kernel-based GP (SMGP) [14], an SSGP [12], an SE kernel-based GP (SEGP) [3], a long short-term memory (LSTM) recurrent neural network [41], and a very recent Transformer-based time-series prediction model, Informer [42]. The SMGP and SEGP model hyperparameters are optimized using a gradient-descent-based approach with the default parameter settings suggested in <https://people.orie.cornell.edu/andrew/code>. The GSM and SM kernel use the same number of kernel components as the GSMP kernel ($Q = 500$ or $Q = 100 \times P$) for a fair comparison. The SSGP model has 500 basis functions with hyperparameters determined using a conjugate-gradient method. The LSTM model has a standard architecture with three hidden layers, each with 100 hidden units while the Informer model follows the default configuration as specified in <https://github.com/zhouhaoyi/Informer2020/tree/main>.

Table III shows the prediction performance of various models as measured by the mean-squared-error (MSE). These results demonstrate that the proposed GSMPPG consistently outperforms their competitors in prediction MSE across all datasets. Notably, GSMPPG outperforms the original GSMGP, particularly in multidimensional datasets, aligning with our findings in Section VI-A. While classical models like LSTM, and more recent models like Informer, require long-term time-series data to learn the underlying patterns effectively, our GSMPPG model excels by building the correlations between series data and minimizing the negative marginal log-likelihood to optimize the kernel hyperparameters. Furthermore, our GSMPPG model outperforms both SMGP and SSGP, as well as other GP models that use basic kernels like SEGP, highlighting its superior performance compared with competitors. In addition to superior prediction performance, our proposed method enhances scalability and communication-efficiency through distributed learning

TABLE IV

PERFORMANCE OF SLIM-KL ($N = 2$ AND $\Delta = 0.01$) WITH VARYING NUMBER OF COMPUTING UNITS s , IN TERMS OF THE PREDICTION MSE. THE LOWEST MSE VALUE FOR EACH DATASET IS HIGHLIGHTED IN BOLD

Dataset	$s = 1$	$s = 4$	$s = 10$	$s = 50$	$s = 100$
ECG	1.2E-02	1.2E-02	1.2E-02	1.5E-02	1.5E-02
CO2	4.9E-01	3.7E-01	6.2E-01	1.1E+00	6.6E-01
Unemployment	2.3E+03	2.0E+03	2.2E+03	2.2E+03	3.3E+03
ALE	2.5E-02	2.5E-02	2.5E-02	2.5E-02	2.5E-02
CCCP	1.6E+01	1.6E+01	1.6E+01	1.6E+01	1.6E+01
Toxicity	1.4E+00	1.4E+00	1.4E+00	1.4E+00	1.4E+00
Concrete	5.9E+01	5.9E+01	5.9E+01	5.9E+01	5.9E+01
Water	1.6E-04	1.6E-04	1.6E-04	1.6E-04	1.6E-04
Wine	4.6E-01	4.6E-01	4.6E-01	4.6E-01	4.6E-01

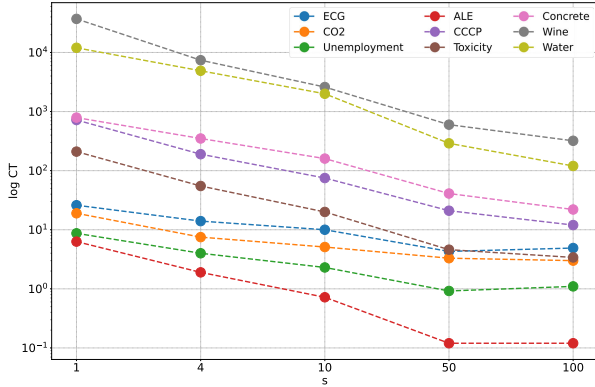


Fig. 3. Total CT (in log scale) for one computing unit, with respect to different values of s .

and quantization, as detailed in Sections VI-C and VI-D, respectively.

The improved prediction performance not only comes from the flexible model capacity of GSMPGP but also benefits from the inherent sparsity in the solutions it generates. The sparsity level of the solutions, calculated as the ratio of nonzero elements to the total number of elements, is reported in Table III. We note that both GSMPGP and GSMGP obtain sparse solutions, in comparison to SMGP. By pinpointing only the significant subkernels, GSMPGP and GSMGP avoid over-parameterization, yielding better generalization capability and lower prediction MSE.

C. Scalability

In this section, we aim to investigate the scalability of SLIM-KL through experiments with varying numbers of computing units and agents. We first consider varying $s \in \{1, 4, 10, 50, 100\}$, and evaluate the performance based on the prediction MSE and computation time (CT). Results in Table IV show that the prediction MSE does not always worsen as s increases. In fact, for several datasets, an improved MSE can be achieved when using more than one computing unit. Moreover, Fig. 3 highlights the advantage of using multiple computing units in reducing the overall CT, supporting the claim given in Remark 1.

We also test the effect of varying the number of local agents on the prediction performance of SLIM-KL. In particular, we set $N \in \{2, 4, 8, 10\}$ and compare them with the centralized case which serves as the baseline. The results in Table V illustrate that SLIM-KL performs better on nearly all datasets when using multiple local agents compared to the centralized

TABLE V

PERFORMANCE OF SLIM-KL ($s = 4$ AND $\Delta = 0.01$) WITH VARYING NUMBER OF LOCAL AGENTS N , IN TERMS OF THE PREDICTION MSE. THE LOWEST MSE VALUE FOR EACH DATASET IS HIGHLIGHTED IN BOLD

Dataset	Centralized	$N = 2$	$N = 4$	$N = 8$	$N = 10$
ECG	1.2E-02	1.2E-02	1.3E-02	1.4E-02	2.2E-02
CO2	4.5E-01	3.7E-01	5.3E-01	8.0E-01	5.7E-01
Unemployment	2.3E+03	2.0E+03	4.9E+03	8.2E+03	9.1E+03
ALE	2.5E-02	2.5E-02	2.5E-02	2.5E-02	2.5E-02
CCCP	1.6E+01	1.6E+01	1.6E+01	1.6E+01	1.6E+01
Toxicity	1.4E+00	1.4E+00	1.8E+00	1.7E+00	1.8E+00
Concrete	8.3E+01	5.9E+01	8.4E+01	8.4E+01	8.5E+01
Water	1.6E-04	1.6E-04	1.6E-04	1.6E-04	1.7E-04
Wine	4.6E-01	4.6E-01	4.7E-01	4.9E-01	5.3E-01

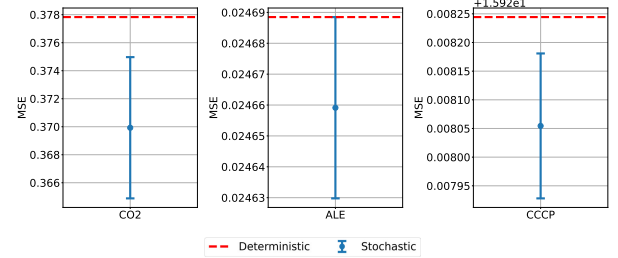


Fig. 4. Performance comparison of SLIM-KL under stochastic quantization versus deterministic quantization, with $\Delta = 0.01$. The blue dots with vertical error bars indicate the mean MSE plus-minus two standard deviations when using stochastic quantization, while the red dashed lines represent the MSE when using deterministic quantization.

case. However, when N is set to be relatively large compared with the size of the training set, the prediction performance degrades as each local agent has fewer data available for training, as evident, e.g., in the *unemployment* dataset. Hence, there is a tradeoff between utilizing more distributed agents and ensuring each agent has sufficient data for training.

To further evaluate the scalability of SLIM-KL, we test its performance on a large dataset. Specifically, we consider the CCCP dataset and increase the training set size $|\mathcal{D}|$ to 9500 and the test set size $|\mathcal{D}_*|$ to 68. We set $N = 10$ and $s = 4$ while keeping the other experimental setup unchanged. Remarkably, we observed that SLIM-KL continues to perform well even with a dataset nearly ten times larger. The prediction MSE remains low at $1.6E + 01$, highlighting the algorithm's ability to handle big data.

D. Effect of Quantization

This section studies the effect of quantization on the performance of SLIM-KL through ablation experiments. We compare two different choices of quantization schemes: stochastic quantization and deterministic quantization. Due to the random nature of the stochastic quantization, we repeat each experiment five times and report the mean MSE with corresponding error bars. Fig. 4 reveals that the stochastic quantization scheme achieves a consistently lower MSE than deterministic quantization, despite its variability. The error bars, denoting plus-minus two standard deviations, suggest that while the MSE for stochastic quantization varies, it does so within a bounded variance, indicating a reliable performance with potential benefits from its inherent randomness.

We further studied the performance of SLIM-KL under different quantization resolutions $\Delta \in \{0.001, 0.01, 0.1, 1, 5\}$ and compared them with the case without quantization. To evaluate

TABLE VI
PERFORMANCE OF SLIM-KL ($s = 4$ AND $N = 2$) WITH AND WITHOUT QUANTIZATIONS IN TERMS OF THE PREDICTION MSE.
THE LOWEST MSE VALUE IS HIGHLIGHTED IN BOLD

Dataset	Without Quantization	With Quantization				
		$\Delta = 0.001$	$\Delta = 0.01$	$\Delta = 0.1$	$\Delta = 1$	$\Delta = 5$
ECG	1.2E-02	1.2E-02	1.2E-02	2.7E-02	1.6E-01	3.9E-01
CO2	3.7E-01	3.7E-01	3.7E-01	6.5E-01	2.7E-01	6.3E-01
Unemployment	2.0E+03	2.0E+03	2.0E+03	2.0E+03	2.0E+03	7.2E+03
ALE	2.5E-02	2.5E-02	2.5E-02	2.5E-02	2.5E-02	0.9E-01
CCCP	1.6E+01	1.6E+01	1.6E+01	1.6E+01	1.6E+01	1.7E+01
Toxicity	1.4E+00	1.4E+00	1.4E+00	1.4E+00	1.4E+00	1.5E+00
Concrete	5.9E+01	5.9E+01	5.9E+01	5.9E+01	5.9E+01	5.9E+01
Water	1.6E-04	1.6E-04	1.6E-04	1.6E-04	4.4E-01	4.4E-01
Wine	4.6E-01	4.6E-01	4.6E-01	4.6E-01	4.6E-01	4.8E-01

TABLE VII
PERFORMANCE COMPARISON ON LONG TIME-SERIES FORECASTING TASKS. THE LOWEST MSE VALUE IS HIGHLIGHTED IN BOLD, AND THE SECOND LOWEST IS UNDERLINED. THE RESULTS OF THE SOTA MODELS ARE REPORTED FROM [43]

Dataset	GSMPGP (ours)	PatchTST (2023)	SegRNN (2023)	MICN (2023)	TiDE (2023)
ETTh1	0.431	0.447	<u>0.434</u>	0.770	0.454
ETTh2	<u>0.382</u>	0.379	0.394	0.956	0.419
ETTm1	0.407	0.416	<u>0.410</u>	0.481	0.413
ETTm2	<u>0.341</u>	0.362	0.330	0.502	0.352
Weather	0.313	0.314	0.310	<u>0.311</u>	0.313

the performance, we empirically computed the prediction MSE and the number of bits required to transmit the local hyperparameters. Assuming double precision, the size of the communication overhead for transmitting a normal vector $\theta^t \in \mathbb{R}^Q$ is $B = 64Q$ bits, while for the quantized vector $\mathcal{Q}(\theta^t)$ it is $B_Q = Q \log_2((\theta_{\max}^t - \theta_{\min}^t)/\Delta + 1)$ bits. Thus, the saving ratio in transmission bits when using quantization can be computed as follows: $B/B_Q = 64/\log_2((\theta_{\max}^t - \theta_{\min}^t)/\Delta + 1)$.

Results in Table VI demonstrate that quantization does not necessarily degrade prediction performance. In fact, on the CO2 dataset, setting $\Delta = 1$ yields the lowest prediction MSE while saving $4.8\times$ fewer bits on average, as depicted in Fig. 5. This improved performance corroborates the design of SLIM-KL, where the enhanced sparsity can be beneficial in guiding the algorithm toward a more favorable optimum, as elaborated in Remark 2. Overall, quantization demonstrates remarkable communication cost savings of over 200 times in the Water dataset while maintaining promising prediction performance.

Remark 3: A lower quantization resolution (e.g., $\Delta = 0.01$) offers higher precision but requires more bits, increasing communication overhead. In contrast, a higher resolution (e.g., $\Delta = 1$) reduces data transmission size, but may degrade performance due to information loss. We found that a resolution of 0.1 strikes a practical balance between model performance and communication cost.

E. Long Time-Series Forecasting

In this section, we extend the evaluation of our proposed method to larger scale datasets for real-world long time-series forecasting tasks. We assess the performance using five popular real-world datasets: *Weather*¹ and four *ETT*² datasets (*ETTh1*, *ETTh2*, *ETTm1*, and *ETTm2*). The details of these datasets can be found in [42]. For the GSMPGP model, we optimized the hyperparameters using SLIM-KL with $s = 100$ computing units, $N = 100$ agents, and a

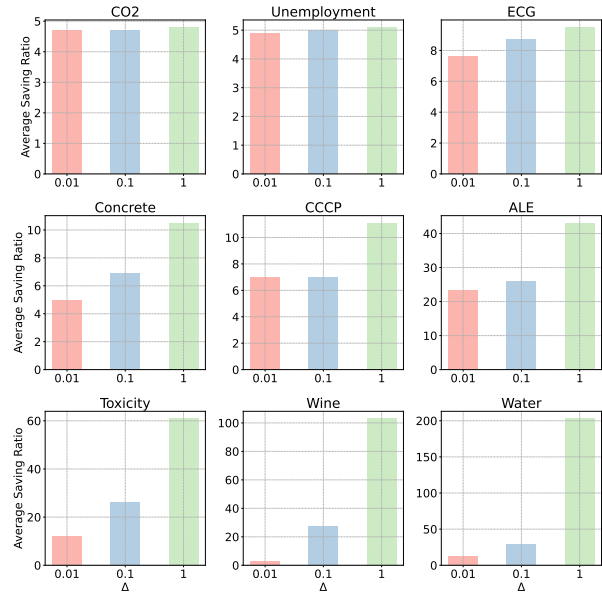


Fig. 5. Average saving ratio in transmitting the local hyperparameters when using quantization versus without quantization, with respect to different quantization resolution Δ .

quantization resolution of $\Delta = 0.01$. For the other models, we followed the configurations reported in [43]. We compare our method against several recent state-of-the-art (SOTA) models, including SegRNN [43], PatchTST [44], MICN [45], and TiDE [46].

The results in Table VII reveal that our GSMPGP model ranked in the top two positions for four out of five datasets, securing first place in two instances. This performance highlights its superiority over other baseline models. Specifically, GSMPGP demonstrated exceptional performance on the *ETT* datasets, nearly achieving SOTA results in terms of MSE. Although there was a slight decrease in performance on the *Weather* dataset, GSMPGP remained competitive compared with other models. These results also indicate that, as we

¹<https://www.bgc-jena.mpg.de/wetter/>

²<https://github.com/zhouhaoyi/ETDataset>

TABLE VIII

PERFORMANCE COMPARISON ON LONG TIME-SERIES FORECASTING TASKS. THE LOWEST MSE VALUE IS HIGHLIGHTED IN BOLD

Dataset	SLIM-KL	DOMKL	CoKle	BoKle
ETTh1	0.431	0.521	0.451	0.441
ETTh2	0.382	0.487	0.391	0.395
ETTm1	0.407	0.463	0.423	0.410
ETTm2	0.341	0.401	0.371	0.359
Weather	0.313	0.372	0.348	0.330

increase the number of agents, SLIM-KL scales effectively, maintaining competitive performance even with a large number of distributed agents. We further compare SLIM-KL with a recent distributed multiple kernel learning method, DOMKL [47], as well as two random feature-based multiple kernel learning methods, CoKle and BoKle [48]. The results summarized in Table VIII demonstrate that SLIM-KL outperforms the other multiple kernel learning methods in terms of prediction MSE.

VII. CONCLUSION

In this article, we proposed a novel GP kernel and a generic sparsity-aware distributed learning framework for LMK learning. Experiments showed that the GSMP kernel, with a modest number of hyperparameters, retains good approximation capability for multidimensional data. The GSMGP with SLIM-KL exhibited superior prediction performance over competitors across diverse datasets. Scalability tests revealed that SLIM-KL maintains and improves prediction performance with increased computing units and agents, demonstrating the framework's effectiveness for distributed environments. Furthermore, incorporating a stochastic quantization scheme significantly reduced communication overhead while preserving performance. These results demonstrate how the proposed method expands the capabilities of GPs to handle large-scale, multidimensional datasets and broadens their applicability across various engineering tasks.

APPENDIX A

SPECTRAL DENSITY OF GSMP KERNEL

We prove Theorem 1. By taking the Fourier transform of the GSMP kernel given in (10), its spectral density can be obtained as follows:

$$\begin{aligned}
 S_{\text{GSMP}}(\omega) &= \int_{\mathbb{R}^P} k(\tau) \exp\{-j 2\pi \tau^\top \omega\} d\tau \\
 &= \int_{\mathbb{R}^P} \sum_{q=1}^Q \theta_q \prod_{p=1}^P k_q(\tau^{(p)}) \exp\{-j 2\pi \tau^\top \omega\} d\tau \\
 &= \sum_{q=1}^Q \theta_q \prod_{p=1}^P \underbrace{\left[\int_{\mathbb{R}} k_q(\tau^{(p)}) \exp\{-j 2\pi \tau^{(p)} \omega^{(p)}\} d\tau^{(p)} \right]}_{\text{Fourier transform of } k_q(\tau^{(p)}): \mathcal{F}[k_q(\tau^{(p)})]}. \quad (27)
 \end{aligned}$$

Substituting the definition of $k_q(\tau^{(p)})$ yields

$$\begin{aligned}
 &\mathcal{F}[k_q(\tau^{(p)})] \\
 &= \mathcal{F}\left[\exp\left\{-2\pi^2 \tau^{(p)^2} v_q^{(p)}\right\} \cos(2\pi \tau^{(p)} \mu_q^{(p)})\right]
 \end{aligned}$$

$$\begin{aligned}
 &= \mathcal{F}\left[\exp\left\{-2\pi^2 \tau^{(p)^2} v_q^{(p)}\right\}\right] * \mathcal{F}[\cos(2\pi \tau^{(p)} \mu_q^{(p)})] \\
 &= \frac{1}{\sqrt{2\pi v_q^{(p)}}} \exp\left\{-\frac{\omega^{(p)^2}}{2v_q^{(p)}}\right\} * \frac{1}{2} (\delta(\omega^{(p)} - \mu_q^{(p)}) \\
 &\quad + \delta(\omega^{(p)} + \mu_q^{(p)})) \\
 &= \frac{1}{2} \left[\frac{1}{\sqrt{2\pi v_q^{(p)}}} \exp\left\{-\frac{(\omega^{(p)} - \mu_q^{(p)})^2}{2v_q^{(p)}}\right\} \right. \\
 &\quad \left. + \frac{1}{\sqrt{2\pi v_q^{(p)}}} \exp\left\{-\frac{(\omega^{(p)} + \mu_q^{(p)})^2}{2v_q^{(p)}}\right\} \right] \\
 &= \frac{1}{2} [\mathcal{N}(\omega^{(p)} | \mu_q^{(p)}, v_q^{(p)}) + \mathcal{N}(\omega^{(p)} | -\mu_q^{(p)}, v_q^{(p)})]. \quad (28)
 \end{aligned}$$

Therefore, the spectral density of the GSMP kernel is given by

$$\begin{aligned}
 S_{\text{GSMP}}(\omega) &= \sum_{q=1}^Q \theta_q \prod_{p=1}^P \mathcal{F}[k_q(\tau^{(p)})] \\
 &= \frac{1}{2^P} \sum_{q=1}^Q \theta_q \prod_{p=1}^P [\mathcal{N}(\omega^{(p)} | \mu_q^{(p)}, v_q^{(p)}) \\
 &\quad + \mathcal{N}(\omega^{(p)} | -\mu_q^{(p)}, v_q^{(p)})] \quad (29)
 \end{aligned}$$

which is a Gaussian mixture.

APPENDIX B

PROOF OF THEOREM 2

Proof: The proof is an extension of [49, Th. 2]. We first state the following lemma that is necessary for the final result.

Lemma 2 (Section IV-A [49]): The term $\mathbf{y}^\top \mathbf{C}^{-1}(\boldsymbol{\theta}) \mathbf{y}$ equals a constant C over all $\boldsymbol{\theta}$ satisfying the n linear constraints $\mathbf{A}\boldsymbol{\theta} = \mathbf{b}$ where

$$\mathbf{A} \triangleq [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_Q], \quad \mathbf{b} \triangleq \mathbf{y} - \sigma_e^2 \mathbf{u} \quad (30)$$

with $\mathbf{v}_q \triangleq \mathbf{K}_q \mathbf{u}$ and $\mathbf{u}$ is any fixed vector such that $\mathbf{y}^\top \mathbf{u} = C$.

Consider the following optimization problem:

$$\begin{aligned}
 &\arg \min_{\boldsymbol{\theta}} \log \det(\mathbf{C}(\boldsymbol{\theta})) \\
 &\text{s. t. } \mathbf{A}\boldsymbol{\theta} = \mathbf{b}, \quad \boldsymbol{\theta} \geq \mathbf{0} \quad (31)
 \end{aligned}$$

where $\mathbf{A}$ and $\mathbf{b}$ are as defined in Lemma 2. Under the GSMP kernel constraint, we define $\mathbf{u} = \mathbf{C}^{-1}(\boldsymbol{\theta}) \mathbf{y}$ and deduce $\mathbf{y} - \sigma_e^2 \mathbf{u} = \sum_{q=1}^Q \theta_q \mathbf{K}_q \mathbf{u}$.

According to Lemma 2, the above constraints hold $\mathbf{y}^\top \mathbf{C}^{-1}(\boldsymbol{\theta}) \mathbf{y}$ constant on a closed, bounded convex polytope. Hence, we focus on minimizing the second term of $l(\boldsymbol{\theta})$ in (7) while holding the first term constant to some C . It follows that any local optimum of $l(\boldsymbol{\theta})$ is also a local optimum of (31) with $C = \mathbf{y}^\top \mathbf{u}$. In [50], it is further established that all minima of (31) occur at extreme points and these extreme points are equivalent to basic feasible solutions, i.e., solutions with at most n nonzero values. This implies that all local minima of (31) must be achieved at sparse solutions. $\square$

APPENDIX C

LOCAL PROBLEM REFORMULATION

Since the term $\tilde{l}_i(\xi_{j,i}, \xi_j^\eta)$ in (21) contains a matrix fractional component, we can introduce an auxiliary variable z such that, $\mathbf{y}^\top \mathbf{C}^{-1}(\xi_{j,i}, \xi_{j,-i}^\eta) \mathbf{y} \leq z$, and use the Schur complement condition [23] to obtain an semidefinite programming (SDP) [51] formulation of the problem

$$\begin{aligned} \min_{\xi_{j,i}, z} \quad & z - \nabla_{\xi_{j,i}} h(\xi_j^\eta)^\top \xi_{j,i} + (\lambda_{j,i}^t)^\top (\xi_{j,i} - \theta_i^{t+1}) \\ & + \frac{\rho_j}{2} \|\xi_{j,i} - \theta_i^{t+1}\|_2^2 \\ \text{s.t.} \quad & \begin{bmatrix} \mathbf{C}(\xi_{j,i}, \xi_{j,-i}^\eta) & \mathbf{y} \\ \mathbf{y}^\top & z \end{bmatrix} \succeq \mathbf{0}, \quad \xi_{j,i} \geq \mathbf{0}. \end{aligned} \quad (32)$$

Due to the fact that $\mathbf{C}(\xi_{j,i}, \xi_{j,-i}^\eta)$ is a sum of positive semidefinite terms, the SDP can be reformulated by introducing $Q/s + 1$ rotated quadratic cone constraints [52]

$$\begin{aligned} \min_{\xi_{j,i}, z, \mathbf{w}} \quad & (\mathbf{1}^\top \mathbf{z}) - \nabla_{\xi_{j,i}} h(\xi_j^\eta)^\top \xi_{j,i} + (\lambda_{j,i}^t)^\top (\xi_{j,i} - \theta_i^{t+1}) \\ & + \frac{\rho_j}{2} \|\xi_{j,i} - \theta_i^{t+1}\|_2^2 \\ \text{s.t.} \quad & \|\mathbf{w}_0\|_2^2 \leq z_0 \\ & \|\mathbf{w}_j\|_2^2 \leq \xi_{j,k+(i-1)Q/s} z_k, \quad k \in \{1, 2, \dots, Q/s\} \\ & = \sum_{k=1}^{Q/s} \mathbf{L}_{k+(i-1)Q/s} \mathbf{w}_k \\ & + \left(\sum_{n \neq i} \sum_{k=1}^{Q/s} \xi_{j,k+(n-1)Q/s}^\eta \mathbf{K}_{k+(n-1)Q/s} + \sigma_e^2 \mathbf{I}_n \right)^{\frac{1}{2}} \mathbf{w}_0 \\ & \xi_{j,i} \geq \mathbf{0}, \quad z \geq 0 \end{aligned} \quad (33)$$

where $\mathbf{z} = [z_0, z_1, \dots, z_{Q/s}] \in \mathbb{R}^{Q/s+1}$, $\mathbf{w}_0 \in \mathbb{R}^n$, $\mathbf{w}_k \in \mathbb{R}^{n_j}$ for $k \in \{1, 2, \dots, Q/s\}$, and $\mathbf{L}_k \in \mathbb{R}^{n \times n_k}$ is a low-rank kernel matrix approximation factor which can be obtained using random Fourier features [53] or Nyström approximation [54]. This reformulation enables us to further express the problem as a second-order cone programming (SOCP) problem [52]

$$\begin{aligned} \min_{\xi_{j,i}, z, \mathbf{w}} \quad & (\mathbf{1}^\top \mathbf{z}) - \nabla_{\xi_{j,i}} h(\xi_j^\eta)^\top \xi_{j,i} + (\lambda_{j,i}^t)^\top (\xi_{j,i} - \theta_i^{t+1}) \\ & + \frac{\rho_j}{2} \|\xi_{j,i} - \theta_i^{t+1}\|_2^2 \\ \text{s.t.} \quad & \left\| \begin{bmatrix} 2\mathbf{w}_0 \\ z_0 - 1 \end{bmatrix} \right\|_2 \leq z_0 + 1 \\ & \left\| \begin{bmatrix} 2\mathbf{w}_k \\ z_k - \xi_{j,k+(i-1)Q/s} \end{bmatrix} \right\|_2 \leq z_k + \xi_{j,k+(i-1)Q/s} \\ & k \in \{1, 2, \dots, Q/s\} \\ & \mathbf{y} = \sum_{k=1}^{Q/s} \mathbf{L}_{k+(i-1)Q/s} \mathbf{w}_k \\ & + \left(\sum_{n \neq i} \sum_{k=1}^{Q/s} \xi_{j,k+(n-1)Q/s}^\eta \mathbf{K}_{k+(n-1)Q/s} + \sigma_e^2 \mathbf{I}_n \right)^{\frac{1}{2}} \mathbf{w}_0 \\ & \xi_{j,i} \geq \mathbf{0}, \quad z \geq 0. \end{aligned} \quad (34)$$

Notice that the term $(\lambda_{j,i}^t)^\top \theta_i^{t+1}$ is a constant, and the two-norm term in the objective, $\|\xi_{j,i} - \theta_i^{t+1}\|_2^2$, can be replaced

with a rotated quadratic cone constraint by introducing a variable $v \geq 0$

$$\begin{aligned} \min_{\xi_{j,i}, z, \mathbf{w}, v} \quad & (\mathbf{1}^\top \mathbf{z}) + \left(\lambda_{j,i}^t - \nabla_{\xi_{j,i}} h(\xi_j^\eta) \right)^\top \xi_{j,i} + \frac{\rho_j}{2} v \\ \text{s.t.} \quad & \left\| \begin{bmatrix} 2\mathbf{w}_0 \\ z_0 - 1 \end{bmatrix} \right\|_2 \leq z_0 + 1 \\ & \left\| \begin{bmatrix} 2\mathbf{w}_k \\ z_k - \xi_{j,k+(i-1)Q/s} \end{bmatrix} \right\|_2 \leq z_k + \xi_{j,k+(i-1)Q/s} \\ & k \in \{1, 2, \dots, Q/s\} \\ & \mathbf{y} = \sum_{k=1}^{Q/s} \mathbf{L}_{k+(i-1)Q/s} \mathbf{w}_k \\ & + \left(\sum_{n \neq i} \sum_{k=1}^{Q/s} \xi_{j,k+(n-1)Q/s}^\eta \mathbf{K}_{k+(n-1)Q/s} + \sigma_e^2 \mathbf{I}_n \right)^{\frac{1}{2}} \mathbf{w}_0 \\ & 2v \frac{1}{2} \geq \sum_{k=1}^{Q/s} \left(\xi_{j,k+(i-1)Q/s} - \theta_{k+(i-1)Q/s}^{t+1} \right)^2 \\ & \xi_{j,i} \geq \mathbf{0}, \quad z \geq 0 \end{aligned} \quad (35)$$

which is equivalent to the conic problem given in (22).

APPENDIX D

PROOF OF THEOREM 3

Proof: To prove the convergence of the DSCA algorithm, in general, we need to select an appropriate step size, which is crucial to ensure the algorithm's convergence, as mentioned in [24]. However, in our case, a step size is no longer necessary as each surrogate function constructed based on (20) already makes $\ell(\xi_{j,i}, \xi_j^\eta)$ a global upper bound for the local objective function. In particular, $\ell(\xi_{j,i}, \xi_j^\eta)$ is a majorization function for the local objective function in the i th block, $L(\xi_{j,i}, \xi_{j,-i}^\eta) \triangleq \ell(\xi_{j,i}, \xi_{j,-i}^\eta) + (\lambda_{j,i}^t)^\top (\xi_{j,i} - \theta_i^{t+1}) + (\rho_j/2) \|\xi_{j,i} - \theta_i^{t+1}\|_2^2$, satisfying the following conditions.

Assumption 2: Each $\ell(\xi_{j,i}, \xi_j^\eta)$ is a majorization function of $L(\xi_{j,i}, \xi_{j,-i}^\eta)$, for $i \in \{1, \dots, s\}$, and satisfies the following conditions.

- 1) $\ell(\xi_{j,i}^\eta, \xi_j^\eta) = L(\xi_{j,i}^\eta, \xi_{j,-i}^\eta)$, for $\xi_{j,i}^\eta \in \Theta_i$.
- 2) $L(\xi_{j,i}, \xi_{j,-i}^\eta) \leq \ell(\xi_{j,i}, \xi_j^\eta)$, $\forall \xi_{j,i} \in \Theta_i, \xi_j^\eta \in \Theta$.

Consequently, we can deduce that

$$\begin{aligned} L(\xi_{j,i}^{\eta+1}, \xi_{j,-i}^\eta) & \leq \ell(\xi_{j,i}^{\eta+1}, \xi_j^\eta) \leq \ell(\xi_{j,i}^\eta, \xi_j^\eta) \\ & = L(\xi_{j,i}^\eta, \xi_{j,-i}^\eta) \end{aligned} \quad (36)$$

ensuring the sequence $\{L(\xi_{j,i}^{\eta+1}, \xi_{j,-i}^\eta)\}_{\eta \in \mathbb{N}}$, is nonincreasing. Based upon Assumptions 1 and 2 for each $\ell(\xi_{j,i}, \xi_j^\eta)$, (23) is guaranteed to hold as a result of [24, Section II.2]. $\square$

APPENDIX E

PROOF OF LEMMA 1

Proof: By using the definition of stochastic quantization function provided in (15), we have

$$\begin{aligned} \mathbb{E}[\mathcal{Q}(\theta)] & = \tau_j \left(1 - \frac{\theta - \tau_j}{\Delta} \right) + \tau_{j+1} \left(\frac{\theta - \tau_j}{\Delta} \right) \\ & = \tau_j \left(1 - \frac{\theta - \tau_j}{\Delta} \right) + (\tau_j + \Delta) \left(\frac{\theta - \tau_j}{\Delta} \right) \end{aligned}$$

$$\begin{aligned}
&= \tau_j - \tau_j \left(\frac{\theta - \tau_j}{\Delta} \right) + \tau_j \left(\frac{\theta - \tau_j}{\Delta} \right) + \theta - \tau_j \\
&= \theta
\end{aligned} \tag{37}$$

so $\mathcal{Q}(\theta)$ is an unbiased estimator of θ . Next, we can show that

$$\begin{aligned}
&\mathbb{E}[(\theta - \mathcal{Q}(\theta))^2] \\
&= (\theta - \tau_j)^2 \left(1 - \frac{\theta - \tau_j}{\Delta} \right) + (\theta - \tau_{j+1})^2 \left(\frac{\theta - \tau_j}{\Delta} \right) \\
&= \Delta^2 \underbrace{\left(1 - \frac{\theta - \tau_j}{\Delta} \right) \left(\frac{\theta - \tau_j}{\Delta} \right)}_{\leq 1/4} \\
&\leq \frac{\Delta^2}{4}
\end{aligned} \tag{38}$$

by using the fact that the variance of a random variable that takes value on $[\tau_j, \tau_{j+1})$ is bounded by $1/4$ [35]. This completes the proof for Lemma 1. $\square$

REFERENCES

- [1] R. C. Suwandi, Z. Lin, Y. Sun, Z. Wang, L. Cheng, and F. Yin, "Gaussian process regression with grid spectral mixture kernel: Distributed learning for multidimensional data," in *Proc. 25th Int. Conf. Inf. Fusion (FUSION)*, Linköping, Sweden, Jul. 2022, pp. 1–8.
- [2] L. Cheng, F. Yin, S. Theodoridis, S. Chatzis, and T.-H. Chang, "Rethinking Bayesian learning for data analysis: The art of prior and inference in sparsity-aware modeling," *IEEE Signal Process. Mag.*, vol. 39, no. 6, pp. 18–52, Nov. 2022.
- [3] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT Press, 2006.
- [4] Y. Pan, X. Yan, E. A. Theodorou, and B. Boots, "Prediction under uncertainty in sparse spectrum Gaussian processes with applications to filtering and control," in *Proc. 34th Int. Conf. Mach. Learn.*, vol. 70, Sydney, NSW, Australia, Aug. 2017, pp. 2760–2768.
- [5] A. Schmidt, P. Morales-Álvarez, and R. Molina, "Probabilistic attention based on Gaussian processes for deep multiple instance learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 10909–10922, Aug. 2024.
- [6] G. Skolidis and G. Sanguinetti, "Semisupervised multitask learning with Gaussian processes," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 24, no. 12, pp. 2101–2112, Dec. 2013.
- [7] K. Lin et al., "TAG: Teacher-advice mechanism with Gaussian process for reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 12419–12433, Sep. 2024.
- [8] S. Theodoridis, *Machine Learning: A Bayesian and Optimization Perspective*, 2nd ed., New York, NY, USA: Academic, 2020.
- [9] G. Chowdhary, H. A. Kingravi, J. P. How, and P. A. Vela, "Bayesian nonparametric adaptive control using Gaussian processes," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 26, no. 3, pp. 537–550, Mar. 2015.
- [10] Y. Xu, F. Yin, W. Xu, J. Lin, and S. Cui, "Wireless traffic prediction with scalable Gaussian process: Framework, algorithms, and verification," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1291–1306, Jun. 2019.
- [11] J. Zhang, H. Ning, X. Jing, and T. Tian, "Online kernel learning with adaptive bandwidth by optimal control approach," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 5, pp. 1920–1934, May 2021.
- [12] M. Lázaro-Gredilla, J. Quiñero-Candela, C. E. Rasmussen, and A. R. Figueiras-Vidal, "Sparse spectrum Gaussian process regression," *J. Mach. Learn. Res.*, vol. 11, no. 63, pp. 1865–1881, Mar. 2010.
- [13] Y. Cui, W. Shi, H. Yang, C. Shao, L. Peng, and H. Li, "Probabilistic model-based reinforcement learning unmanned surface vehicles using local update sparse spectrum approximation," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 1283–1293, Feb. 2024.
- [14] A. Wilson and R. Adams, "Gaussian process kernels for pattern discovery and extrapolation," in *Proc. Int. Conf. Mach. Learn.*, Atlanta, GA, USA, 2013, pp. 1067–1075.
- [15] F. Yin, L. Pan, T. Chen, S. Theodoridis, Z. T. Luo, and A. M. Zoubir, "Linear multiple low-rank kernel based stationary Gaussian processes regression for time series," *IEEE Trans. Signal Process.*, vol. 68, pp. 5260–5275, 2020.
- [16] K. Chen et al., "Recent advances in data-driven wireless communication using Gaussian processes: A comprehensive survey," *China Commun.*, vol. 19, no. 1, pp. 218–237, Jan. 2022.
- [17] H. Liu, Y.-S. Ong, X. Shen, and J. Cai, "When Gaussian process meets big data: A review of scalable GPs," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 11, pp. 4405–4423, Nov. 2020.
- [18] P. Zhai and R. T. Rajan, "Distributed Gaussian process hyperparameter optimization for multi-agent systems," in *Proc. ICASSP - IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Rhodes Island, Greece, May 2023, pp. 1–5.
- [19] Z. Dang, B. Gu, C. Deng, and H. Huang, "Asynchronous parallel large-scale Gaussian process regression," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 8683–8694, Jun. 2024.
- [20] F. Yin et al., "FedLoc: Federated learning framework for data-driven cooperative localization and location data processing," *IEEE Open J. Signal Process.*, vol. 1, pp. 187–215, 2020.
- [21] M. Titsias, "Variational learning of inducing variables in sparse Gaussian processes," in *Proc. 12th Int. Conf. Artif. Intell. Statist.*, 2009, pp. 567–574.
- [22] H. Liu, Y.-S. Ong, and J. Cai, "Large-scale heteroscedastic regression via Gaussian process," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 2, pp. 708–721, Feb. 2021.
- [23] S. P. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [24] G. Scutari and Y. Sun, "Parallel and distributed successive convex approximation methods for big-data optimization," in *Multi-Agent Optimization* Cham, Switzerland: Springer, 2018, pp. 141–308.
- [25] "The MOSEK optimization toolbox for MATLAB manual. Version 10.0," MOSEK ApS, Copenhagen, Denmark, 2022.
- [26] Y. Sun, P. Babu, and D. P. Palomar, "Majorization-minimization algorithms in signal processing, communications, and machine learning," *IEEE Trans. Signal Process.*, vol. 65, no. 3, pp. 794–816, Feb. 2017.
- [27] K. N. Plataniotis and D. Hatzinakos, "Gaussian mixtures and their applications to signal processing," in *Advanced Signal Processing*. Boca Raton, FL, USA: CRC Press, 2001.
- [28] A. Kashyap, T. Başar, and R. Srikant, "Quantized consensus," *Automatica*, vol. 43, no. 7, pp. 1192–1203, Jul. 2007.
- [29] M. M. Amiri, D. Gunduz, S. R. Kulkarni, and H. V. Poor, "Federated learning with quantized global model updates," 2020, *arXiv:2006.10672*.
- [30] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, Jan. 2011.
- [31] S. Zhu and B. Chen, "Quantized consensus by the ADMM: Probabilistic versus deterministic quantizers," *IEEE Trans. Signal Process.*, vol. 64, no. 7, pp. 1700–1713, Apr. 2016.
- [32] Y. Zhao, F. Yin, F. Gunnarsson, M. Amirijoo, E. Özkan, and F. Gustafsson, "Particle filtering for positioning based on proximity reports," in *Proc. Int. Conf. Inf. Fusion (FUSION)*, Washington, DC, USA, Jul. 2015, pp. 1046–1052.
- [33] D. Jin, A. M. Zoubir, F. Yin, C. Fritsche, and F. Gustafsson, "Dithering in quantized RSS based localization," in *Proc. IEEE 6th Int. Workshop Comput. Adv. Multi-Sensor Adapt. Process. (CAMSAP)*, Cancun, Mexico, Dec. 2015, pp. 245–248.
- [34] T. T. Doan, S. T. Maguluri, and J. Romberg, "Fast convergence rates of distributed subgradient methods with adaptive quantization," *IEEE Trans. Autom. Control*, vol. 66, no. 5, pp. 2191–2205, May 2021.
- [35] J.-J. Xiao and Z.-Q. Luo, "Decentralized estimation in an inhomogeneous sensing environment," *IEEE Trans. Inf. Theory*, vol. 51, no. 10, pp. 3564–3575, Oct. 2005.
- [36] M. Grant and S. Boyd, "CVX: MATLAB software for disciplined convex programming, version 2.1," Stanford Univ., Stanford, CA, USA, 2014.
- [37] M. Hong, Z.-Q. Luo, and M. Razaviyayn, "Convergence analysis of alternating direction method of multipliers for a family of nonconvex problems," *SIAM J. Optim.*, vol. 26, no. 1, pp. 337–364, Jan. 2016.
- [38] W. Deng and W. Yin, "On the global and linear convergence of the generalized alternating direction method of multipliers," *J. Scientific Comput.*, vol. 66, no. 3, pp. 889–916, Mar. 2016.
- [39] B. Wohlberg, "ADMM penalty parameter selection by residual balancing," 2017, *arXiv:1704.06209*.
- [40] D. García, "Robust smoothing of gridded data in one and higher dimensions with missing values," *Comput. Statist. Data Anal.*, vol. 54, no. 4, pp. 1167–1178, Apr. 2010.
- [41] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

- [42] H. Zhou et al., “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, May 2021, pp. 11106–11115.
- [43] S. Lin, W. Lin, W. Wu, F. Zhao, R. Mo, and H. Zhang, “SegRNN: Segment recurrent neural network for long-term time series forecasting,” 2023, *arXiv:2308.11200*.
- [44] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, May 2022, pp. 1–12.
- [45] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, and Y. Xiao, “MICN: Multi-scale local and global context modeling for long-term series forecasting,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, May 2023, pp. 1–14.
- [46] A. Das, W. Kong, A. Leach, S. Mathur, R. Sen, and R. Yu, “Long-term forecasting with TiDE: Time-series dense encoder,” in *Proc. Trans. Mach. Learn. Res.*, Aug. 2023, pp. 1–21.
- [47] S. Hong and J. Chae, “Distributed online learning with multiple kernels,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 3, pp. 1263–1277, Mar. 2023.
- [48] S. Hong, “Online multikernel learning method via online biconvex optimization,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 16630–16643, Nov. 2024.
- [49] D. Wipf and B. D. Rao, “Sparse Bayesian learning for basis selection,” *IEEE Trans. Signal Process.*, vol. 52, no. 8, pp. 2153–2164, Jul. 2004.
- [50] D. G. Luenberger, *Linear and Nonlinear Programming*. Reading, MA, USA: Addison-Wesley, Jan. 1984.
- [51] L. Vandenberghe and S. Boyd, “Semidefinite programming,” *SIAM Rev.*, vol. 38, no. 1, pp. 49–95, Mar. 1996.
- [52] M. S. Lobo, L. Vandenberghe, S. Boyd, and H. Lebret, “Applications of second-order cone programming,” *Linear Algebra Appl.*, vol. 284, nos. 1–3, pp. 193–228, Nov. 1998.
- [53] A. Rahimi and B. Recht, “Random features for large-scale kernel machines,” in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 20, 2007, pp. 1177–1184.
- [54] C. Williams and M. Seeger, “Using the Nyström method to speed up kernel machines,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Denver, CO, USA, Nov. 2000, pp. 682–688.



Richard Cornelius Suwandi (Graduate Student Member, IEEE) received the B.Sc. degree in statistics from The Chinese University of Hong Kong, Shenzhen, China, in 2023, where he is currently pursuing the Ph.D. degree in computer and information engineering with the School of Science and Engineering.

His research interests include Bayesian optimization, probabilistic machine learning, and generative models.



Zhidi Lin (Graduate Student Member, IEEE) received the M.Sc. degree in communication and information systems from Xiamen University, Xiamen, China, in 2019, and the Ph.D. degree in computer and information engineering from the School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, China, in 2024.

He is currently a Post-Doctoral Research Fellow with the Department of Statistics and Data Science, National University of Singapore, Singapore. His research interests include statistical signal processing, Bayesian deep learning, approximate Bayesian inference, and data assimilation.



Feng Yin (Senior Member, IEEE) received the B.Sc. degree from Shanghai Jiao Tong University, Shanghai, China, in 2008, and the M.Sc. and Ph.D. degrees from Technische Universitaet Darmstadt, Darmstadt, Germany, in 2011 and 2014, respectively.

From 2014 to 2016, he was a Post-Doctoral Researcher with Ericsson Research, Linköping, Sweden. Since 2016, he has been with The Chinese University of Hong Kong, Shenzhen, China, where he is currently an Assistant Professor with the School of Science and Engineering. He has published more than 100 top-tier journals and conference papers, and 20 patents/standards. His research interests include statistical signal processing, Bayesian learning and optimization, and sensory data fusion.

Dr. Yin is a member of the IEEE Machine Learning and Signal Processing (MLSP) Technical Committee and the IEEE SPS Scholarship Committee. He was a recipient of the Chinese Government Award for Outstanding Self-Financed Students Abroad in 2013 and the Marie Curie Young Fellowship from the European Union in 2014. He was the Finalist for the IEEE CAMSAP Best Paper Award in 2013 and received the Best Paper Award of ICSINC Conference in 2022. He has served as an Associate Editor for the *Signal Processing* (Elsevier) and currently serving as an Associate Editor for IEEE TRANSACTIONS ON SIGNAL PROCESSING.



Zhiguo Wang (Member, IEEE) received the Ph.D. degree from Sichuan University, Chengdu, China, in 2018.

From 2018 to 2020, he was a Post-Doctoral Researcher with The Chinese University of Hong Kong, Shenzhen, China. Since 2020, he has been working as an Associate Professor with the College of Mathematics, Sichuan University. His current research interests are in the fields of nonconvex optimization, optimal transport, geometric deep learning, and various optimization problems from image processing and big data applications.



Sergios Theodoridis (Life Fellow, IEEE) is currently a Professor Emeritus of signal processing and machine learning with the Department of Informatics and Telecommunications, National and Kapodistrian University of Athens, Athens, Greece. He has authored the book “*Machine Learning: From the Classics to Deep Networks, Transformers, and Diffusion Models*” (Academic Press, 3rd Edition, 2025), the co-author of the best-selling book “*Pattern Recognition*” (Academic Press, 4th Edition, 2009), and the co-author of the book “*Introduction to Pattern Recognition: A MATLAB Approach*” (Academic Press, 2010). He is the co-author of seven papers that have received best paper awards including the 2014 IEEE Signal Processing Magazine Best Paper Award and the 2009 IEEE Computational Intelligence Society Transactions on Neural Networks Outstanding Paper Award.

Dr. Theodoridis is a fellow of Institution of Engineering and Technology (IET) and European Association for Signal Processing (EURASIP), and a Corresponding Fellow of the Royal Society of Edinburgh (RSE). He has served as the Vice President for the IEEE Signal Processing Society, the President for the EURASIP, and a member of the Board of Governors for the IEEE Circuits and Systems (CAS) Society.